BACHELOROPPGAVE

Grafmodellering av OpenOwnerShip og utvikling av FastAPI

Julian Normann Valle

Munazza Maqbool Hussain

Scott Mejlænder-Larsen

Institutt for Informasjonsteknologi
Postadresse: Postboks 4 St. Olavs plass, 0130 Oslo
Besøksadresse: Holbergs plass, Oslo
Telefon: 22 45 32 00

PROSJEKT NR.
1

BACHELORPROSJEKT

HOVEDPROSJEKTETS TITTEL	**DATO**
Grafmodellering av OpenOwnership for Politiets IT-enhet og utvikling av FastAPI.	19.05.2026
	ANTALL SIDER / BILAG
	127
PROSJEKTDELTAKERE Hussain, Munazza Maqbool Mejlænder-Larsen, Scott Valle, Julian Normann	**INTERN VEILEDER** Gjester, Torunn

OPPDRAGSGIVER	**KONTAKTPERSON**
Politiets IT-enhet	Stabell-Kulø, Tage

SAMMENDRAG

Prosjektet går ut på å utvikle et system som leser, parser og importerer data fra OpenOwnership Register til en Neo4j grafdatabase. Målet er å modellere personer og selskaper som noder, og eierforhold som relasjoner (kanter), slik at komplekse eierkjeder kan analyseres med Cypher-spørringer. Systemet skal tilby et API for å utføre forhåndsdefinerte spørringer mot grafdatabasen og returnere resultatene.

3 STIKKORD Neo4j
Python
FastAPI

Forord

Denne rapporten markerer avslutningen på våre bachelorgrader i informasjonsteknologi, anvendt datateknologi og dataingeniør ved OsloMet våren 2026. Rapporten beskriver prosessen og løsningen vi har implementert i samarbeid med Politiets IT-enhet.

Vi fikk kontakt med Politiets IT-enhet på eget initiativ fra gruppens side. Vi tok kontakt med Politiets IT-enhet via deres nettside, og ble deretter satt i kontakt med seksjonsleder for seksjon Data og Analyse i avdeling Utvikling, Design og Data. Gjennom dialog med Politiets IT-enhet fikk vi presentert en konkret problemstilling. Parallelt var vi i kontakt med andre virksomheter og mottok ulike prosjektforslag. Etter en samlet vurdering valgte vi å samarbeide med Politiets IT-enhet, da prosjektet fremstod som både faglig relevant og utfordrende.

Prosjektet har gitt oss muligheten til å kjenne på hvordan det er å jobbe som analytikere og utviklere, med reelle behov. Det føles meningsfullt å ha gjort noe som faktisk skal være til hjelp for oppdragsgiver og som kan bidra til å avdekke skjulte relasjoner og sammenhenger i store datamengder.

Vi ønsker å rette en stor takk til Politiets IT-enhet for å ha vært åpne, tilgjengelige og engasjerte gjennom hele prosessen. Takk også til vår interne veileder ved OsloMet, Torunn Gjester, for faglige innspill og veiledning underveis. Til slutt vil vi takke hverandre i gruppen for innsatsen, samarbeidet og engasjementet som har gjort prosjektet mulig.

Oslo, mai 2026

Julian Normann Valle, Munazza Maqbool Hussain, Scott Mejlænder-Larsen.

Innholdsfortegnelse:

Innholdsfortegnelse: 4
Figurliste: 8
1 Introduksjon 10
1.1 Gruppemedlemmer 10
1.2 Veiledere 10
1.3 Oppdragsgiver 11
1.4 Bakgrunn og motivasjon 11
1.4.1 OpenOwnership 12
1.4.2 Politiets behov 12
1.5 Prosjektbeskrivelse 12
1.5.1 Prosjektet 12
1.5.2 Problemstilling 14
1.5.3 Målgruppe 14
2 Bakgrunn 15
2.1 Begreper 15
2.2 Teknologier 16
2.2.1 Neo4j 16
2.2.2 Cypher 18
2.2.3 Rust (ytelse og import) 19
2.2.4 FastAPI 20
2.2.4.1 Hva er et API? 20
2.2.4.2 Hva er FastAPI? 20
2.2.4.3 Hvorfor FastAPI ble valgt 21
2.2.4.4 Bruk av FastAPI i prosjektet 21
2.2.4.5 API-endepunkter og HTTP-metoder 21
2.2.4.6 Dokumentasjon og testing 22
2.3 Rammeverk og verktøy 22
2.3.1 Programmeringsspråk (Python) 22
2.3.2 Biblioteker og avhengigheter 23
2.3.3 Utviklingsmiljø (Visual Studio Code) 25
2.3.4 Versjonskontroll (Git og GitHub) 26

2.3.4.1 Git....26
2.3.4.2 GitHub....27
2.3.5 Infrastruktur (VM / Pegasus)....28
3 Kravspesikasjon....29
3.1 Funksjonelle krav....29
3.2 Ikke-funksjonelle krav....29
3.2.1 Robusthet og stabilitet....29
3.2.2 Reproduserbarhet og dokumentasjon....30
3.2.3 Sikkerhet....30
3.2.4 Ytelse og skalerbarhet....30
3.2.5 Kvalitet og brukervennlighet....30
3.3 Tekniske rammer....30
4 Systemarkitektur og design....32
4.1 Overordnet arkitektur....32
4.2 Datamodell (Neo4j)....33
4.3 Database-design....37
4.4 API-design....38
4.5 Autentisering og logging....40
4.6 Implementasjon av infrastruktur....43
5 Prosessdokumentasjon....45
5.1 Metodisk tilnærming....45
5.1.1 Utviklingsmetode....45
5.1.2 Teknisk metode....46
5.1.3 Testmetode....48
5.1.4 Dokumentasjon og kommunikasjon....50
5.1.5 Prosjektfaser....51
6 Implementasjon / Produktdokumentasjon....53
6.1 Datainnhenting og parsing....53
6.1.1 Datasettbeskrivelse....53
6.1.2 Parsing og forbehandling....55
6.2 Import til Neo4j....57
6.2.1 Datamodell (noder)....57
6.2.2 Relasjoner....58
6.2.3 Cypher og importlogikk....59
6.2.4 Batcher og Ytelse....60
6.2.5 Importprosessen....61

6.3 API-utvikling (FastAPI)....65
6.3.1 Responsmodeller og validering....68
6.3.2 Kommunikasjon med Neo4j....69
6.3.3 Eierskap og kontroll....69
6.3.4 Etterforskningsrelasjoner....70
6.4 Klient / API-bruk....71
6.4.1 Innlogging og token....71
6.4.2 Eksempel på forespørsel....71
6.5 Logging og sikkerhet....72
6.5.1 Autentisering....72
6.5.2 Brukerdatabase og passord....72
6.5.3 Tilgangskontroll....73
6.5.4 Logging....73
7 Testdokumentasjon....75
7.1 Teststrategi....75
7.2 Valideringstest....75
7.3 Importkontroll....77
7.4 Ytelsestest....79
7.5 Aggregeringstest....81
7.6 Ytelsestest av importprosessen....84
8 Brukerdokumentasjon....88
8.1 Systemoversikt....88
8.2 Installasjon og oppsett....88
8.2.1 Oppstart av Neo4j....88
8.2.2 Oppstart av API....89
8.3 Bruk av systemet....89
8.3.1 Import av data....89
8.3.2 Bruk av API....90
8.3.3 Innlogging og autentisering....90
8.3.4 Eksempel på respons....92
8.3.5 Status-endepunkt....92
8.4 Bruk av Neo4j Browser....93
8.5 Feilhåndtering....94
8.6 Begrensninger....94
9 Avslutning....95
9.1 Diskusjon....97

9.2 Konklusjon..... 98
9.3 Videre arbeid..... 98
Kildeliste.....100
Vedlegg.....103

Figurliste:

Figur 1: Eksempel på node og relasjon i grafdatabase....17
Figur 2: Rust-Logo....19
Figur 3: FastAPI-Logo Kilde https://fastapi.tiangolo.com/img/logo-margin/logo-teal.png... 20
Figur 4: Python-Logo Kilde https://www.python.org/static/img/python-logo.png....22
Figur 5: Forsiden av Visual Studio Code....25
Figur 6: Git-Logo Kilde https://git-scm.com/images/logo@2x.png....26
Figur 7: GitHub-logo Kilde https://brand.github.com/_next/static/media/logo-03.cc5e5332.png....27
Figur 8: Figuren viser ooc....33
Figur 9: Modell av databasen laget ved hjelp av draw.io kilde drawio.com....35
Figur 10: De tilhørende tabellene til databasen laget ved hjelp av draw.io kilde drawio.com.36
Figur 11: Eierskapsrelasjonen....37
Figur 12: Persontilkoblinger....39
Figur 13: API-endepunkter....40
Figur 14: Innlogging vellykket....40
Figur 15: Innlogging mislykket....41
Figur 16: Unauthorized....41
Figur 17: Authorized....42
Figur 18: API-logger....42
Figur 19: Eksempel fra loggen til Admin-brukeren....43
Figur 20: Test.py....48
Figur 21: Skjermbilde fra Dadroit JSON Viewer....54
Figur 22: Skjermbilde av et statement i Dadroit JSON Viewer....55
Figur 23: Innlesning av data....56
Figur 24: personStatement....57
Figur 25: Cypher-Spørringer for Person nodene....60
Figur 26: Nettverks konfigurasjonen til Neo4....61
Figur 27: Bolt tilkobling innstillingene til Neo4j....62
Figur 28: HTTP-tilkobling innstillingene til Neo4j....62
Figur 29: Minneinnstillingene til Neo4j....62
Figur 30: Noder or Relasjoner....64
Figur 31: Eksempel på noder og koblinger i Neo4j....65
Figur 32: Neo4j-tilkobling....66
Figur 33: API-endepunkter....68

Figur 34: Persondetaljer Funksjon..... 69
Figur 35: Uthenting av personopplysninger i APIet..... 71
Figur 36: Resultatet av uthentingen av personopplysninger..... 72
Figur 37: Eksempel på loggfilstruktur..... 74
Figur 38: Detaljert logg..... 74
Figur 39: Bilde av Cypher-spørring for valideringstest..... 76
Figur 40: PersonStatement peker mot Person..... 76
Figur 41: Databaseverifisering..... 77
Figur 42: Node Labels..... 78
Figur 43: Relationship types..... 78
Figur 44: Indeks-kontroll..... 78
Figur 45: Sirkularitetsdeteksjon..... 79
Figur 46: Plan 1..... 80
Figur 47: Plan 2..... 80
Figur 48: Plan 3..... 80
Figur 49: Aggregeringstest med tid..... 82
Figur 50: Aggregeringstest med tabell..... 82
Figur 51: Resultat topp 10 funn..... 83
Figur 52: Cypher-spørring for eierskapsforhold..... 83
Figur 53: Import av noder med 8 arbeidere..... 84
Figur 54: Import av Noder med 12 arbeidere..... 85
Figur 55: Ressursbruk Nodeimport med 12 arbeidere..... 85
Figur 56: Høyt diskbruk..... 86
Figur 57: Normalt diskbruk..... 86
Figur 58: Relasjonsimporten..... 87
Figur 59: Neo4j browser etter oppstart..... 89
Figur 60: POST /token, innlogging i Browser..... 91
Figur 61: Innlogging med autogenerert Authorization..... 91
Figur 62: Formattering på svar på API-et..... 92
Figur 63: Respons fra Status endepunktet..... 92
Figur 64 : Et lite utvalg av noder..... 93

1 Introduksjon

Dette kapittelet gir en introduksjon til prosjektet og etablerer rammene for arbeidet. Det presenterer prosjektgruppen, veiledere og oppdragsgiver, samt redegjør for bakgrunn og motivasjon for prosjektet.

Videre gis det beskrivelse av oppgaven, inkludert problemstilling og målgruppe. Kapittelet har i mål om å gi leseren en tydelig forståelse av prosjektet.

1.1 Gruppemedlemmer

Gruppen består av tre studenter, alle med forskjellig studieretning. Dette har gitt oss stor dekning i kompetanse og ferdigheter. Gruppen består av følgende medlemmer:

Munazza Maqbol Hussain – Dataingeniør

Julian Normann Valle – Anvendt datateknologi

Scott Mejlænder-Larsen – Informasjonsteknologi

1.2 Veiledere

Internveileder hos OsloMet

Torunn Gjester
torunngj@oslomet.no

1.3 Oppdragsgiver

Oppdragsgiver er politiets IT-enhet (PIT). PIT er en organisasjon i politiet som har som hovedoppgave å støtte politiets arbeid gjennom bruk av teknologi som virkemiddel (Politiet, u. å). Enheten har ansvar for utvikling, drift, forvaltning og brukerstøtte av politiets IT-systemer og tjenester (Wikipedia, 2026).

Oppdragsgiver

Politiets IT-enhet

Telefon: 22 06 82 00

E-post: post.it-enhet@politiet.no

Adresse: Fridtjof Nansens vei 14, 0369 Oslo

Kontaktperson hos Arbeidsgiver

Tage Stabell Kulø

Telefonnummer: +4748178042

E-post: tage.stabell-kulo@politiet.no

1.4 Bakgrunn og motivasjon

Bachelorgruppen hadde et ønske om å jobbe med et prosjekt som har tydelig samfunnsrelevans og praktisk verdi. Prosjektet fra PIT ble derfor valgt, da dette er relevant for studiet og samtidig et reelt behov for politiets arbeid.

Prosjektet ville også gi gruppen mulighet til å samarbeide med en profesjonell aktør med høy kompetanse innen data og teknologi. Dette har bidratt til økt læringsutbytte og verdifull erfaring med relevante teknologier og systemer.

1.4.1 OpenOwnership

OpenOwnership Register (OO) er et internasjonalt selskap som har et mål om å øke åpenheten rundt eierskap i selskaper. OO samler inn data, standardiserer det og publiserer det som åpent datasett. Datasettene er offentlig tilgjengelige og inneholder informasjon om personer, selskaper og ulike eier- og selskapsforhold på tvers av land og jurisdiksjoner (OpenOwnership, u.å.). Dataene beskriver indirekte og direkte eierskap, og gjør det mulig å se hvordan eierskap er strukturert gjennom flere ledd.

1.4.2 Politiets behov

Politiet har behov for å analysere komplekse nettverk bestående av personer og selskaper, i arbeid for bekjemping av økonomisk kriminalitet. Ved slik etterforskning, er det viktig å forstå hvem som eier eller kontrollerer et selskap. Dette er utfordrende, spesielt når eierskap kan gå på tvers av landegrenser og gjennom flere ledd. Slike strukturer kan brukes for å skjule reelle eiere og flytte verdier uten tilsyn. Selv om datasett fra OO gir verdifull innsikt i slike eierskapstrukturer, er det vanskelig å bruke det direkte i etterforskning, på grunn av størrelse og kompleksitet. Derfor er det behov for en løsning som kan strukturere dataene og gjøre analysen enklere.

1.5 Prosjektbeskrivelse

1.5.1 Prosjektet

Prosjektet går ut på å analysere og strukturere et datasett fra OpenOwnership Register. Datasettet er svært stort og omfattende, med størrelse på ca. 38 GB og over 36 millioner linjer

med JSON-data. Hver linje representerer en entitet, person, selskap eller en relasjon mellom disse. Dette gjør at datasettet er vanskelig å analysere direkte, spesielt som relasjonsdatabase. I en relasjonsdatabase må slike relasjoner håndteres gjennom JOIN-operasjoner mellom tabeller. I et stort datasett, med mange koblinger mellom entiteter, vil slike spørringer bli kompliserte og ineffektive. For å gjøre dette enklere blir datasettet transformert til grafdatabase gjennom Neo4j. I en grafdatabase representeres data i form av noder og relasjoner. Dette gjør det enklere og mer effektivt å utføre spørringer som inneholder flere koblinger. Politiet bruker python, og ønsker derfor at løsningen vår bruker dette programmeringsspråket.

Prosjektet består av to deler:

- En databehandling og lagringsdel, hvor datasettet konverteres fra relasjonsdatabase til grafdatabase i Neo4j. Dette gjør det mulig å representere komplekse forbindelser mellom personer og selskaper på en strukturert måte.

- En applikasjonsdel med FastAPI, som gjør det mulig å arbeide med data fra Neo4j. Dette inkluderer søk etter personer og visning av eierskap og relasjoner. Løsningen inkluderer autentisering og logging.

Pipeline for prosjektet:

JSON (OpenOwnership)
->
Parsing (Python)
->
Transformasjon
->
Neo4j (grafdatabase)
->

FastAPI

->

Klient

Denne løsningen vil gjøre det mulig for politiet å analysere store mengder data på en effektiv måte. Grafdatabasen gjør at etterforskere enklere kan få oversikt og identifisere skjulte forbindelser og potensielle antydninger til økonomisk kriminalitet.

1.5.2 Problemstilling

Problemstilling:
Hvordan kan data fra OpenOwnership Register modelleres som grafdatabase i Neo4j, med noder, relasjoner og attributter, og samtidig gjøres tilgjengelig gjennom en applikasjon. Slik at eier- og kontrollstrukturer mellom personer og selskaper kan avdekkes og analyseres på en effektiv måte?

1.5.3 Målgruppe

Prosjektets målgruppe er ansatte og etterforskere i politiet som arbeider med analyse av økonomisk kriminalitet. Løsningen er laget for å støtte deres arbeid, ved å gi dem en bedre oversikt over relasjonene mellom aktørene.

2 Bakgrunn

Dette kapittelet presenterer det teoretiske grunnlaget og de teknologiske løsningene som danner fundamentet for prosjektet. Først beskrives sentrale begreper knyttet til grafdatabaser og analyse av relasjoner mellom entiteter. Deretter presenteres teknologiene som ble benyttet i utviklingen av løsningen, med fokus på grafdatabasen Neo4j, spørrespråket Cypher og rammeverket FastAPI. Kapittelet gir dermed et faglig grunnlag for de metodiske og tekniske valgene som beskrives senere i rapporten.

2.1 Begreper

For å forstå den tekniske løsningen i prosjektet er det nødvendig å definere sentrale begreper knyttet til grafdatabaser og analyse av eierskapsstrukturer.

Grafdatabase:

En grafdatabase (GDB) er en type database som bruker grafer med noder og relasjoner til å representere store mengder data. Denne strukturen gjør det mulig med spørring og visualisering av ulike forhold mellom datapunkter, noe som gjør den nyttig for applikasjoner med svært sammenkoblede data (Neo4j, 2025).

Node:

En node representerer en entitet i databasen. Dette kan være alt fra en person, et selskap til en adresse. Hver node inneholder egenskaper som beskriver entiteten nærmere.

Relasjon:

En relasjon er en forbindelse mellom to noder. For eksempel kan en person være knyttet til et selskap gjennom relasjonen EIER. Relasjonen kan også inneholde egne egenskaper som beskriver forbindelsen mer detaljert.

Egenskap:

Egenskaper er informasjon lagret som nøkkel-verdi-par. Eksempelvis for en person kan dette være navn, fødselsdato eller nasjonalitet. Egenskaper gjør det mulig å lagre strukturert informasjon om både noder og relasjoner. (Neo4j, u.å.-c)

Kontrollkjede:

En kontrollkjede beskriver hvordan eierskap kan spores gjennom flere ledd. Dersom selskap A eier selskap B, og selskap B eier selskap C, kan selskap A indirekte ha kontroll over selskap C. Slike sammenhenger er sentrale i analyse av selskapsstrukturer.

2.2 Teknologier

I prosjektet ble det benyttet flere teknologier for å modellere, lagre og analysere data. De viktigste teknologiene var Neo4j og spørringsspråket Cypher.

2.2.1 Neo4j

Neo4j er en grafdatabase utviklet for å håndtere datasett med komplekse forbindelser mellom entiteter. (Neo4j, u.å.-a) I motsetning til relasjonsdatabaser, hvor relasjoner mellom tabeller

beregnes gjennom join-operasjoner, lagrer Neo4j relasjoner direkte mellom nodene. Dette gir raskere og mer effektiv analyse av sammenhenger i store nettverk. (Neo4j, u.å.-b)

I Neo4j består datastrukturen av:

1. Noder, som representerer entiteter.
2. Relasjoner, som representerer forbindelser.
3. Egenskaper, som lagrer informasjon.
4. Labels, som kategoriserer noder.
   (Neo4j, u.å.-c)

Et eksempel på en enkel modell kan være en person som bor i en bestemt by:

```
CREATE (p: Person {navn: "Jon"})

CREATE (l: Lokasjon {by: "Drammen"})

CREATE (p)-[:BOR_I {type: "hjem"}]->(l)
```

I Neo4j vil dette vises figur (1)



Figur 1: Eksempel på node og relasjon i grafdatabase.

Denne strukturen gjør det enklere å visualisere og analysere komplekse eierskapsforhold på en mer effektiv og intuitiv måte enn ved bruk av tradisjonelle databaser.

I dette prosjektet ble Neo4j valgt i samråd med oppdragsgiver, samtidig som problemstillingen krevde analyse av relasjoner mellom selskaper, eiere og tilknyttede personer. Grafmodellen gjorde det enklere å identifisere både direkte og indirekte forbindelser.

2.2.2 Cypher

Cypher er det spørrespråket som brukes i Neo4j for å opprette, hente og analysere data. Cypher er et deklarativt språk, noe som betyr at brukeren beskriver hva som skal hentes fremfor hvordan databasen skal utføre operasjonen. (Neo4j, u.å.-c)

I prosjektet ble Cypher brukt til:

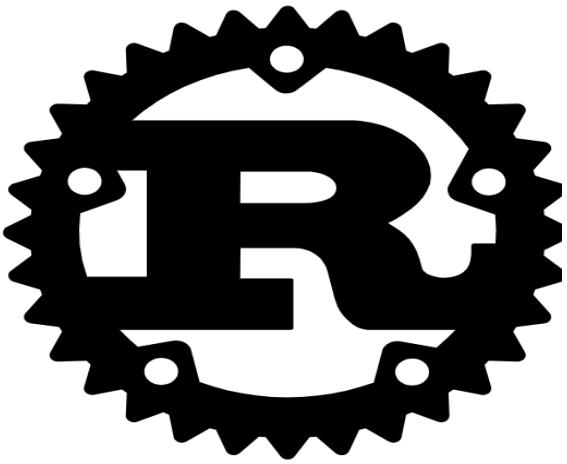
1. opprettelse av noder.
2. opprettelse av relasjoner.
3. import av datasett.
4. søk etter eierskapsforhold.
5. identifisering av indirekte kontroll.
6. analyse av nettverk mellom selskaper.

Et eksempel på en spørring som finner personer som eier selskaper:

```
MATCH (p:Person)-[:EIER]->(s:Selskap)
RETURN p, s
```

Cypher var sentralt i prosjektet fordi det gjorde det mulig å analysere komplekse forbindelser i datasettet på en effektiv og lesbar måte.

2.2.3 Rust (ytelse og import)



Figur 2: Rust-Logo
kilde https://rust-lang.org/static/images/rust-logo-blk.svg

Neo4j sin Python driver er ikke optimal for å parse data inn i Neo4j på en effektiv måte. Det er en del lettere å få økt effektiviteten av parsingen ved bruk av andre programmeringsspråk som for eksempel Java, C, C++ og lignende (Bauer, 2024).

På bakgrunn av dette ble det undersøkt muligheter for hvordan ytelsen kunne forbedres ved import til Neo4j. Målet var å redusere tiden som ble brukt til å parse og innlaste det store datasettet.

Dette førte til vurderingen av en Rust-basert Neo4j-driver. Programmeringsspråket Rust er kjent for å ha høy ytelse og minneffektivitet. Det uoffisielle hobbyprosjektet Neo4j-rust-driver implementerer kommunikasjon med Neo4j via Bolt-protokollen. Dette gjør det mulig for Rust-programmer å kommunisere direkte med Neo4j-databasen.

Ifølge prosjektets dokumentasjon kan Rust-driveren føre til betydelige ytelsesforbedringer sammenlignet med den tradisjonelle Python-driveren. I enkelte tilfeller kan det bli opptil 10 ganger raskere (robsdedude, u.å.).

Dette gjør at Rust-programmer kan kommunisere direkte med Neo4j-databasen og det er opptil 10 ganger raskere enn bare den vanlige Neo4j-Python-driveren (robsdedude, u.å.).

2.2.4 FastAPI



Figur 3: FastAPI-Logo
Kilde: https://fastapi.tiangolo.com/img/logo-margin/logo-teal.png

2.2.4.1 Hva er et API?

Application programming interfaces (APIer) er et sett med regler som gjør det mulig for programmer å kommunisere sammen. For eksempel, når en bruker logger inn på en nettside, brukes det et API som sender brukernavnet og passordet til serveren (backend). Deretter returnerer serveren et innloggingstoken hvis det er riktig brukernavn og passord. Hvis passordet er feil blir en feilmelding returnert med at det er feil brukernavn og passord (Microsoft, 2025).

2.2.4.2 Hva er FastAPI?

FastAPI er et moderne og raskt nettrammeverk som er designet for at man lett skal kunne lage APIer ved bruk av Python på en effektiv måte (FastAPI, u.å.-a)

2.2.4.3 Hvorfor FastAPI ble valgt

FastAPI ble valgt med samråd med oppdragsgiver. Grunnen til dette er fordi at FastAPI er spesielt designet og optimalisert til å brukes sammen med programmeringsspråket Python som blir brukt i prosjektet. Det er veldig lett å lære og det går raskt å kode, man får også automatisk API-dokumentasjon (FastAPI, u.å.-a)

2.2.4.4 Bruk av FastAPI i prosjektet

I prosjektet brukes FastAPI til å hente ut data fra Neo4j-databasen. Get-endepunkter brukes til å hente data fra API-et. Disse aktiveres av frontend ved brukerforespørsler og returnerer data fra databasen. Hver av GET-requestene er koblet til Cypher som kommuniserer med databasen og returnerer data tilbake til for eksempel frontend.

2.2.4.5 API-endepunkter og HTTP-metoder

Når man bygger API-er så fins det fire standard HTTP-metoder som gjør spesifikke ting, disse er.

POST: Brukes til når man skal legge til data, for eksempel hvis man skal legge til en person i databasen.

GET: Brukes til å hente ut data ut ifra databasen, for eksempel at man skal hente ut personinformasjon ut ifra databasen.

PUT: Brukes til å oppdatere eksisterende data i databasen fra en verdi til en annen.

DELETE: Delete brukes når man skal slette noe fra databasen, for eksempel en person eller lignende.

Det er viktig å bruke riktig API-endepunkter til riktige oppgaver for å få det til å fungere på en korrekt måte (FastAPI, u.å.-b).

2.2.4.6 Dokumentasjon og testing

FastAPI har også to docs frontend sider der man kan teste ut API-endepunktene for å sjekke om alt er korrekt koblet. Disse sidene er tilbudt fra Swagger UI og ReDoc (FastAPI, u.å.-c).

2.3 Rammeverk og verktøy

2.3.1 Programmeringsspråk (Python)



Figur 4: Python-Logo
Kilde: https://www.python.org/static/img/python-logo.png

I dette prosjektet har gruppen brukt programmeringsspråket Python til utvikling av kodene. Python er et moderne objektorientert høynivåprogrammeringsspråk som blir brukt mye rundt om i verden. Blant annet på grunn av at det er enkelt å skrive og feilsøke kode (Python Software Foundation, u.å.).

2.3.2 Biblioteker og avhengigheter

I prosjektet er det blitt brukt mange Python-biblioteker. Dette er for å forenkle utviklingen og håndtere ulike oppgaver som utvikling av kode, API-et, databehandling og databasekommunikasjon. Bruken av bibliotekene har gjort det mulig å utvikle løsningen mer effektivt samtidig som koden blir mer strukturert og oversiktlig.

Biblioteker som er brukt i prosjektet kan man se i tabellen under.

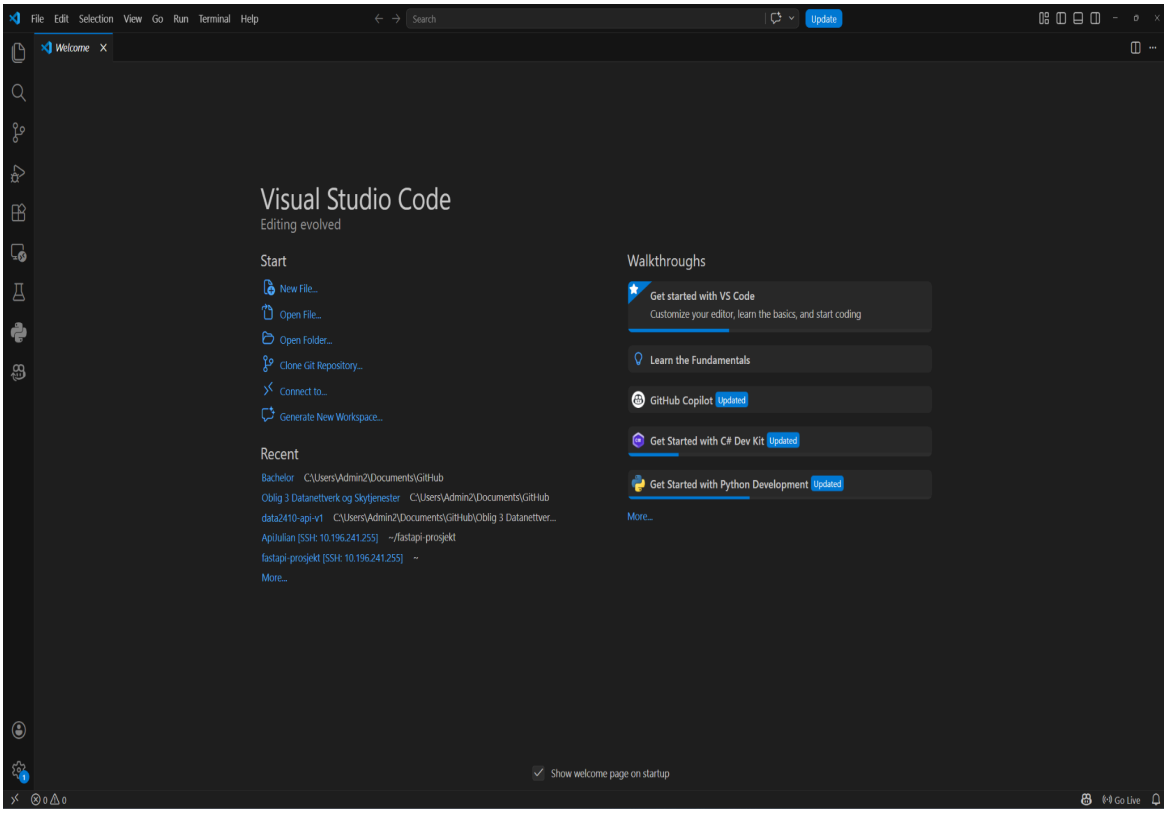
Bibliotek	Beskrivelse
Neo4j (Grafdatabase)	Blir brukt til å kommunisere med Neo4j-databasen
Pydantic	Blir brukt til datavalidering og modellering
python-jose (jwt, JWTError)	Blir brukt til autentisering med JSON Web Tokens
passlib (CryptContext)	Blir brukt til hashing av passord
SQLite3	Blir brukt til lokal lagring av for eksempel brukere, passord og lignende

FastAPI	Blir brukt for å utvikle API-et
Uvicorn	Blir brukt for å starte og kjøre API-serveren
logging	Blir brukt for å logge hva brukere har gjort i API-et
orjson	Blir brukt for rask håndtering av JSON-data
multiprocessing	Blir brukt for parallell prosessering
itertools (kombinasjoner)	Blir brukt for å generere kombinasjoner av data
os / pathlib / time / datetime	Blir brukt til filhåndtering, tidsstyring og systemoperasjoner

2.3.3 Utviklingsmiljø (Visual Studio Code)

I prosjektet ble programmet Visual Studio Code brukt til å utvikle og kjøre kodene. Dette verktøyet ble valgt siden gruppen har opparbeidet seg mye erfaring i løpet av de tre årene på OsloMet og at det er brukervennlig og effektivt. Programmet tilbyr også flere nyttige funksjoner, blant annet feilsøking av kode, mange utvidelser man kan laste ned, og støtte for flere programmeringsspråk (Microsoft, u.å-a).

Under kan man se et skjermbilde fra forsiden til Visual Studio Code figur (5)



Figur 5: Forsiden av Visual Studio Code

2.3.4 Versjonskontroll (Git og GitHub)

2.3.4.1 Git



Figur 6: Git-Logo
Kilde https://git-scm.com/images/logo@2x.png

Git er et distribuert versjonskontrollsystem som tar øyeblikksbilder av filsystemet når man lagrer eller committer ting i prosjektet. Det blir bare tatt øyeblikksbilder av filene som har blitt endret fra forrige versjon for å unngå unødig ressursbruk.

Git sin tankeprosess er at dataene er en strøm av øyeblikksbilder. Om man har lyst til å sjekke historikken av endringer i prosjektet så kan man lett sjekke dette lokalt (Git, u.å).

2.3.4.2 GitHub



Figur 7: GitHub-logo
Kilde: https://brand.github.com/_next/static/media/logo-03.cc5e5332.png

Når man bruker GitHub kan man samarbeide med å lagre, dele og skrive ny kode, plattformen er også skybasert. Samarbeidet foregår i noe som kalles GitHub repository, dette er en felles prosjektlagringsplass for kode. Man kan også bli med i andre repositories som andre har laget.

Erfaringen med GitHub førte til at samarbeidet ble effektivt og det ga en god flyt i prosjektet.

Pull-requests er en viktig funksjon i GitHub som muliggjør at andre samarbeidspartnere kan se over endringene man har gjort før det blir pushet til hovedgrenen i GitHub. Dette bidrar igjen til å oppdage feil og forbedre kodekvaliteten (GitHub, u.å-b).

Det er også mulig å pushe koden direkte til hovedgrenen, men dette anbefales som oftest ikke siden det kan være vanskelig å oppdage sine egne kodefeil. I store prosjekter og selskaper er det vanlig å kreve godkjenning fra mange personer. Dette er en viktig del av kvalitetssikringen av koden. Kodefeil kan få store konsekvenser både sikkerhetsmessige, økonomiske og lignende (GitHub, u.å-a).

2.3.5 Infrastruktur (VM / Pegasus)

En virtuell maskin (VM) er en datamaskin som kjører i et virtuelt miljø i motsetning til datamaskiner som bruker fysisk maskinvare. Dette gjør at det blir mulig å kjøre forskjellige programvarer og tjenester på en isolert og fleksibel måte. Man kan lett tilpasse hvor mye datakraft man trenger i motsetning til en fysisk datamaskin som har en viss mengde datakraft (Microsoft, u.å-b).

Tjenesten Pegasus er en tjeneste som er bygget, vedlikeholdt og opprettholdt av OsloMet-storbyuniversitetet(OsloMet, 2024). Tjenesten bruker frontend fra OpenStack for å opprette og redigere de forskjellige virtuelle maskinene.

En detaljert oversikt om hvordan VMen er satt opp i Pegasus finnes i Vedlegg 1: *VM-oppsett*

3 Kravspesikasjon

3.1 Funksjonelle krav

De funksjonelle kravene beskriver hvilken funksjonalitet systemet skal tilby.

F1: Systemet skal kunne lese og behandle data fra OpenOwnership Register fra fil.
F2: Systemet skal kunne modellere personer og selskaper som noder, og lagre disse i Neo4j grafdatabasen.
F3: Systemet skal legge til rette for visualisering av noder og relasjoner i Neo4j Browser.
F4: Systemet skal kunne utføre spørringer i Cypher som avdekker eier- og kontrollkjeder.
F5: Systemet skal kunne kjøres flere ganger uten å opprette duplikate noder eller relasjoner.
F6: Systemet skal tilby et API for å utføre forhåndsdefinerte spørringer mot grafdatabasen og returnere resultatene.
F7: API-et skal støtte spørringer som avdekker både direkte og indirekte eierskap, uten at brukeren manuelt må benytte Neo4j Browser.
F8: API-et skal tilby et enkelt status-endepunkt som bekrefter at tjenesten kjører og at tilkoblingen til databasen fungerer.
F9 (valgfritt): Systemet kan støtte flere visualiseringstyper gjennom verktøy som NeoDash, for eksempel grafer, tabeller og diagrammer.

3.2 Ikke-funksjonelle krav

De ikke-funksjonelle kravene beskriver kvalitetskrav til systemet, som ytelse, sikkerhet og robusthet.

3.2.1 Robusthet og stabilitet

NF1: Systemet skal være stabilt under behandling av store datamengder.
NF2: Systemet skal være robust mot mangelfulle eller inkonsistente data.

3.2.2 Reproduserbarhet og dokumentasjon

NF3: Systemet skal være reproduserbart, slik at samme input gir samme resultat.
NF4: Systemet skal være tilstrekkelig dokumentert slik at det kan forstås og videreutvikles.
NF5: API-et skal tilby automatisk dokumentasjon gjennom OpenAPI/Swagger via FastAPI.

3.2.3 Sikkerhet

S1: Systemet skal kun benytte åpne og offentlig tilgjengelige data.
S2: Data skal ikke deles eller eksponeres utenfor prosjektets virtuelle maskin.
S3: Systemet skal ikke lagre sensitive personopplysninger utover det som finnes i datasettet.
S4: Feil under import eller behandling skal ikke føre til tap av eksisterende data.
S5: API-et skal kun være tilgjengelig internt på den virtuelle maskinen.

3.2.4 Ytelse og skalerbarhet

Y1: Systemet skal kunne importere et representativt delsett av OpenOwnership Register innen rimelig tid på tilgjengelig maskinvare.
Y2: Analyse-spørringer skal kunne kjøres interaktivt med akseptabel responstid.
Y3: Systemet skal kunne håndtere økende datamengder uten vesentlig reduksjon i ytelse.
Y4: Den virtuelle maskinen skal ha tilstrekkelig minne (RAM) til å håndtere datasettet i Neo4j.

3.2.5 Kvalitet og brukervennlighet

K1: Grafmodellen skal være intuitiv og lett å forstå for analytikere.
K2: Spørringer skal være lesbare og godt dokumentert.
K3: Resultater skal kunne visualiseres på en oversiktlig måte i Neo4j Browser.
K4: Koden skal være strukturert og kommentert for å sikre god vedlikeholdbarhet.

3.3 Tekniske rammer

De tekniske rammene beskriver hvilke teknologier og plattformer som benyttes i løsningen.

T1: Systemet skal benytte Neo4j Community Edition som grafdatabase.
T2: Python skal benyttes til parsing og behandling av data.
T3: Cypher skal benyttes til å opprette noder, relasjoner og utføre spørringer.
T4: Systemet skal kjøres på en Linux-basert virtuell maskin levert via Pegasus.
T5: Det skal benyttes indekser og constraints i Neo4j for å sikre datakvalitet og ytelse.
T6: API-et skal implementeres i Python ved bruk av FastAPI.
T7: API-et skal koble seg til Neo4j via Neo4j sin offisielle Python-driver (Bolt-protokollen).

4 Systemarkitektur og design

Dette kapittelet forteller om den tekniske oppbygningen av løsningen og de viktigste designvalgene som ble gjort. Kapittelet tar først for seg den overordnede arkitekturen og hvordan de forskjellige delene samhandler. Videre forklares datamodellen i Neo4j, databasedesignet og API-designet. Til slutt løsninger for autentisering og logging og implementasjon av infrastrukturen.

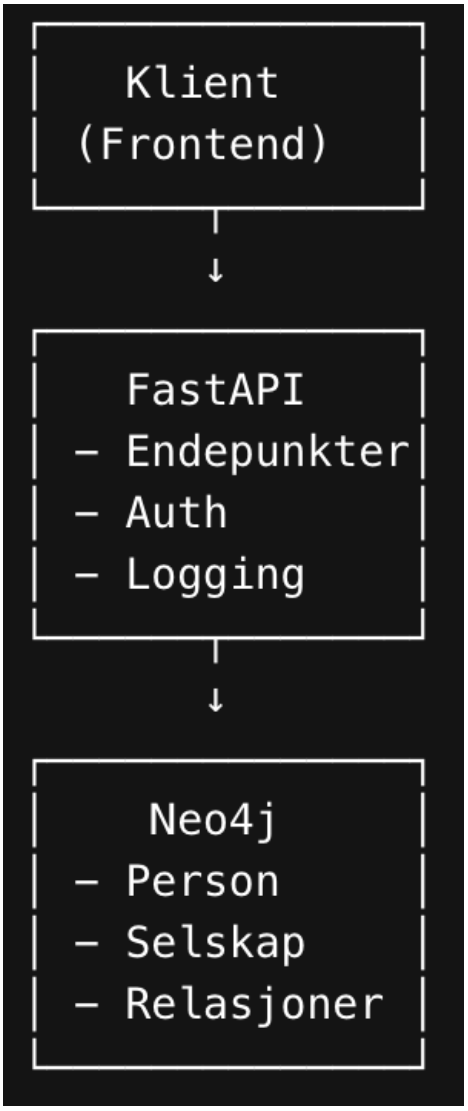
4.1 Overordnet arkitektur

Løsningen er bygget opp på flere lag, bestående av klient, API, grafdatabase og miljø i VM. Klienten brukes til å sende forespørsler til systemet, mens FastAPI fungerer som mellomlaget mellom brukeren og databasen i Neo4j. Her brukes Neo4J som database for lagring og uthenting av data, ved hjelp av spørringer. Dette kjøres på en VM.

FastAPI applikasjonen har i oppgave å behandle forespørsler, validering og autentisering og kommunisere med Neo4j. Denne kommunikasjonen foregår gjennom Pythons «AsyncGraphDatabase» import, som setter opp tilkobling til databasen når applikasjonen starter.

Dataflyten starter med at en bruker sender forespørsel til klienten til API-et. Om forespørselen krever autentisering, kontrolleres brukerens JWT-token før den blir behandlet videre. Deretter kjører API-et Cypher-spørringer mot Neo4j, og returnerer resultatene til klienten.

Systemet bruker også SQLite3 database som brukes i håndtering av brukere, innlogging, logging og forespørsler.



Figur 8: Figuren viser ooc

4.2 Datamodell (Neo4j)

Datasettet, som ble hentet fra OpenOwnership, var opprinnelig formatert som JSON-baserte objekter. Disse objektene er strukturert som statements. Det vil si statements som representerer person (*personStatement*), selskap (*entityStatement*) eller

eierskap/kontrollforhold (*ownershipOrControllStatement*). Her blir informasjon lagret som nøkkel-verdi-par, med egenskaper som navn, identifikator, adresser og datoer.

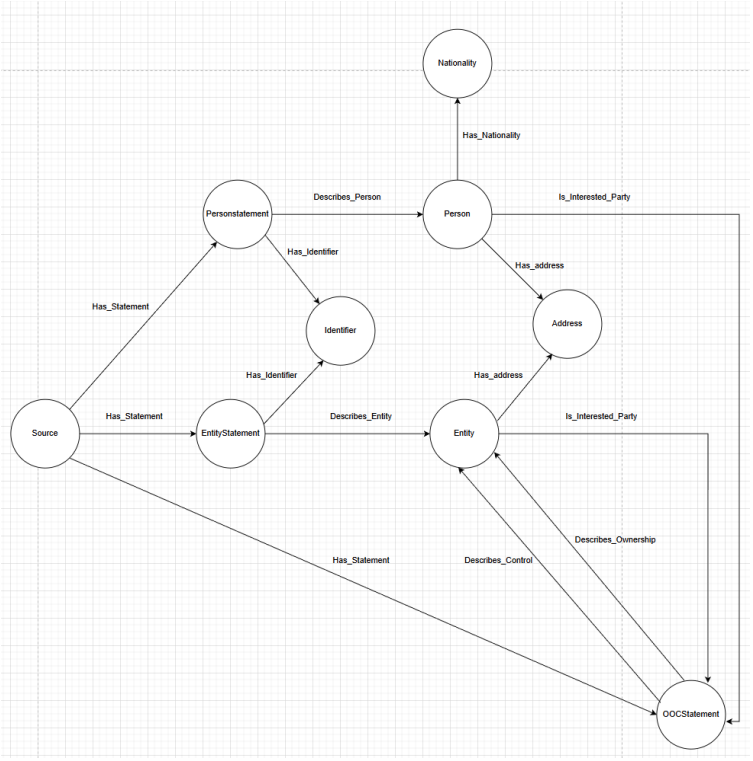
I det opprinnelige datasettet er sammenhengene mellom objektene vanskelige å analysere. Dette er fordi sammenhengene ikke er satt opp som relasjoner, men man må tolke referanser mellom de forskjellige statementene. For å gjøre det enklere å se disse forbindelsene og analysere dataen ble datasettet gjort om til en grafdatabase i Neo4j. Her representeres enheter som noder og forbindelsene mellom dem som relasjoner.

Sentrale noder i grafdatabasen er *Person*, *Entity* og *OOCStatement*. Person representerer en fysisk person og *Entity* er et selskap eller en juridisk enhet. *OOCStatement* blir brukt som en node mellom Person og *Entity,* som beskriver eierskap- og kontrollforhold. I tillegg benyttes noder som *PersonStatement, EntityStatement, Address, Source, Identifier* og *Nationality* som støttenoder, for å lagre tilleggsinformasjon.

Relasjoner beskriver hvordan nodene er koblet sammen. Person er koblet til noden *PersonStatement* igjennom relasjonen *Describes_Person,* mens *EntityStatement* kobles til *Entity* med relasjonen *Describes_entity.*

Eierskap og kontroll representeres med *OOCStatement.* En person eller enhet blir koblet til denne statementen gjennom relasjonen *Is_interested_Party*, og deretter videre til *Entity* gjennom *Describes_ownership* eller *Describes_Control.* Dette gjør at modellen kan uttrykke direkte og indirekte eierskap, hvor det også blir lagret informasjon om eierandel, type interesse og tidsperiode. Relasjonen *Investigation_Link* brukes i etterforskningskontekst, hvor den gjør det mulig å se forbindelser mellom spesifikke personer.

Figuren under viser den overordnede datamodellen som er tatt i bruk i Neo4j. Modellen viser hvordan sentrale noder og relasjoner er strukturert for å vise eierskap og kontrollforhold:



Figur 9: Modell av databasen laget ved hjelp av draw.io
kilde drawio.com

Denne modellen gjør det mulig å følge aktører over flere ledd, noe som er viktig under analyse av komplekse nettverk. Det er totalt 9 nodetyper og 9 relasjonstyper i databasedesignet.

De tilhørende tabellene som forklarer hva nodene inneholder kan man se i figur (10)



Figur 10: De tilhørende tabellene til databasen laget ved hjelp av draw.io
kilde drawio.com

Relasjonen *Describes_Ownership* er den eneste relasjonen som inneholder egne attributter, og er koblet fra *OOCStatement* til *Entity*, denne informasjonen vises i tabellen under.

Describes_Ownership
type startDate share_exact share_minimum share_maximum endDate details

Figur 11: Eierskapsrelasjonen

4.3 Database-design

Dataene lagres i Neo4j som noder med egenskaper og relasjonene mellom dem. Egenskapene hentes direkte fra OpenOwnership datasettet, dette gjelder informasjon som navn, dato, identifikator og andre relevante egenskaper. Ettersom datasettet er svært omfattende, blir det tatt i bruk batch- og parallell-prosessering. Dette ga importen bedre ytelse og mer effektiv innlasting.

Datasettet fra OpenOwnership inneholdt flere referanser til samme entiteter. For å effektivisere database-designet må duplikater fjernes. Dette blir gjort med MERGE kommandoen, som sørger for at nye noder bare blir opprettet dersom den allerede ikke eksisterer:

```
MERGE (p:Person {personID: obj.personID})
```

Denne kommandoen sørger for at det bare er en *PersonID*, uavhengig av hvor mange ganger det oppstår i datasettet.

Det brukes også constraints på de viktigste identifikatorene, for å sikre at det ikke oppstår duplikater:

```
CREATE CONSTRAINT person_personID IF NOT EXISTS
```

```
FOR (p:Person) REQUIRE p.personID IS UNIQUE
```

Duplikater for relasjonene blir behandlet på en annen måte, avhengig av type relasjon. Relasjoner med faste koblinger bruker MERGE, mens relasjoner som kan oppstå flere steder, som i eierskapsforhold, bruker CREATE. Dette gjør at flere relasjoner mellom samme noder kan oppstå parallelt.

4.4 API-design

API-et er satt opp som mellomlaget mellom klienten og Neo4j-databasen. Det håndterer forespørsler, autentisering og kommunikasjon med databasen. Systemet bruker API-et for strukturert tilgang og kontroll av data, i stedet for direkte tilgang til databasen.

API-et er satt opp i flere lag, hvor klienten sender forespørsel til API-et. API-et håndterer validering, autentisering og kommunikasjon med databasen, og til slutt returnerer resultatet tilbake til klienten.

API-et er delt inn i flere funksjonsområder: Endepunkt for brukerhåndtering og autentisering, endepunkt for søk av personer og selskaper og endepunkt for håndtering av etterforskningsrelasjoner. API-et bruker standard HTTP-metoder, hvor GET henter data, POST oppretter nye relasjoner og PATCH/PUT oppdaterer og DELETE sletter data.

Hvert endepunkt er satt opp med input, validering av forespørsel og returnering av svar. Når endepunktet kalles, opprettes det et kall til en tilhørende funksjon, som inneholder Cypher-spørringene. Dette resultatet behandles og returneres til klienten som JSON-responser. Enkelte av endepunktene bruker DEPENDS for å sikre at brukeren er autentisert før forespørselen behandles.

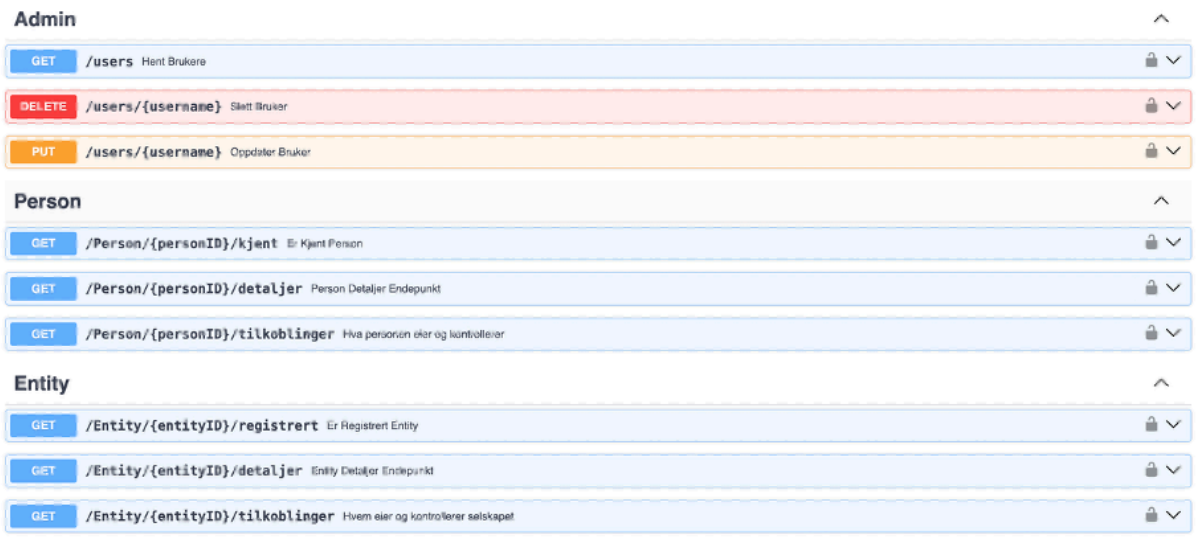
Eksempel på slik struktur er endepunktet for tilkoblinger til en person. Her mottar API-et *PersonID,* sjekker at brukeren er innlogget, henter informasjon fra Neo4j og returnerer til

klienten figur (12)

```python
@app.get(
    "/Person/{personID}/tilkoblinger",
    tags=["Person"],
    response_model=PersonTilkoblingerSvar,
    summary="Hva personen eier og kontrollerer",
)
async def person_tilkoblinger(
    personID: str,
    bruker: str = Depends(hent_innlogget_bruker),
    session=Depends(hent_okt),
):
    """
    - **owns**: selskaper personen eier, med eierandeler
    - **controls**: selskaper personen kontrollerer
    """
    data = await hent_person_tilkoblinger(session, personID)
    if not data:
        raise HTTPException(status_code=404, detail="Person ble ikke funnet")
    logging.getLogger(bruker).info(f"Persontilkoblinger hentet: {personID}")
    return data
```

Figur 12: Persontilkoblinger

Figur 13 viser en oversikt over endepunktene vist i Swagger UI



Figur 13: API-endepunkter

4.5 Autentisering og logging

Prosjektet sitt FastAPI bruker en token-basert autentisering med JWT (Jason Web Tokens). Dette funker ved at brukeren logger inn via token-endepunktet med sitt brukernavn og passord, deretter verifiserer serveren passordet ved bruk av hash og bcrypt. Hvis alt er korrekt blir en JWT-token generert og blir brukt i alle requests, backend må verifisere tokenet før tilgang til API-endepunktene gis. Grunnen til at API-et bruker tokens er på grunn av sikkerhet ved at tokens har en utløpstid, rask validering og at passordet ikke sendes for hver request. I koden blir OAuth2PasswordBearer brukt til å håndtere tokens fra requests, jwt.encode() lagrer tokens og jwt.decode() validerer tokens. Hvis man logger inn med korrekt brukernavn og passord får man kode 200 ok returnert figur (14)

```
INFO:     10.47.242.50:54033 - "POST /token HTTP/1.1" 200 OK
```

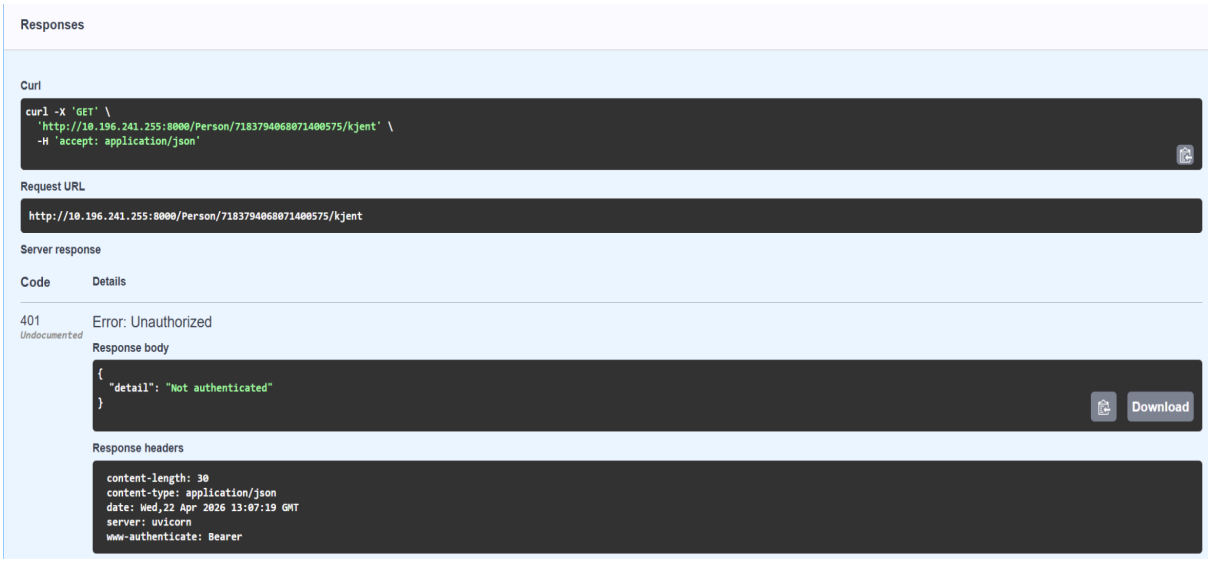
Figur 14: Innlogging vellykket

Hvis man derimot logger inn med feil brukernavn eller passord blir kode 400 Bad Request returnert i backend figur (15).

```
INFO:     10.47.242.50:50052 - "POST /token HTTP/1.1" 400 Bad Request
```

Figur 15: Innlogging mislykket

Om man prøver å kalle et endepunkt uten å ha logget inn med sin bruker får man kode 401 Unauthorized figur (16)



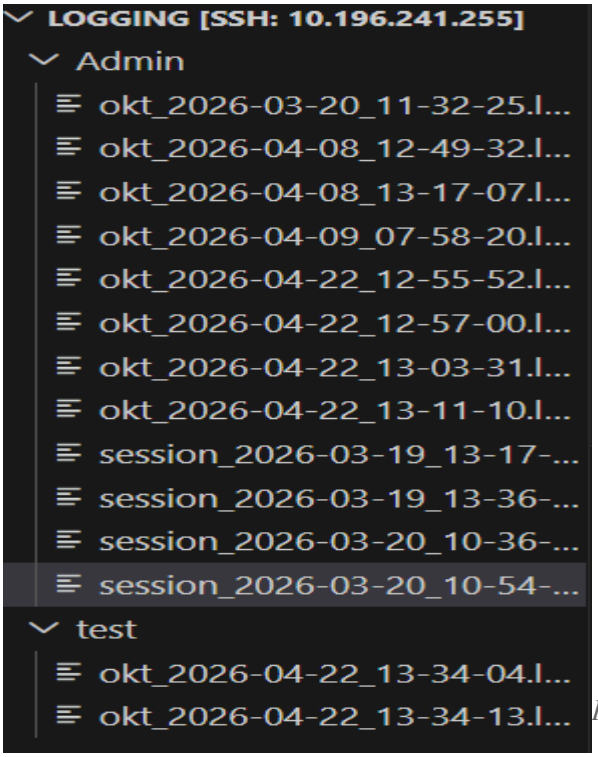
Figur 16: Unauthorized

Om man deretter logger inn med sin bruker og prøver på nytt får man en respons kode (200). Dette er siden det er en Authorization Bearer til stede som har med en korrekt token figur (17)



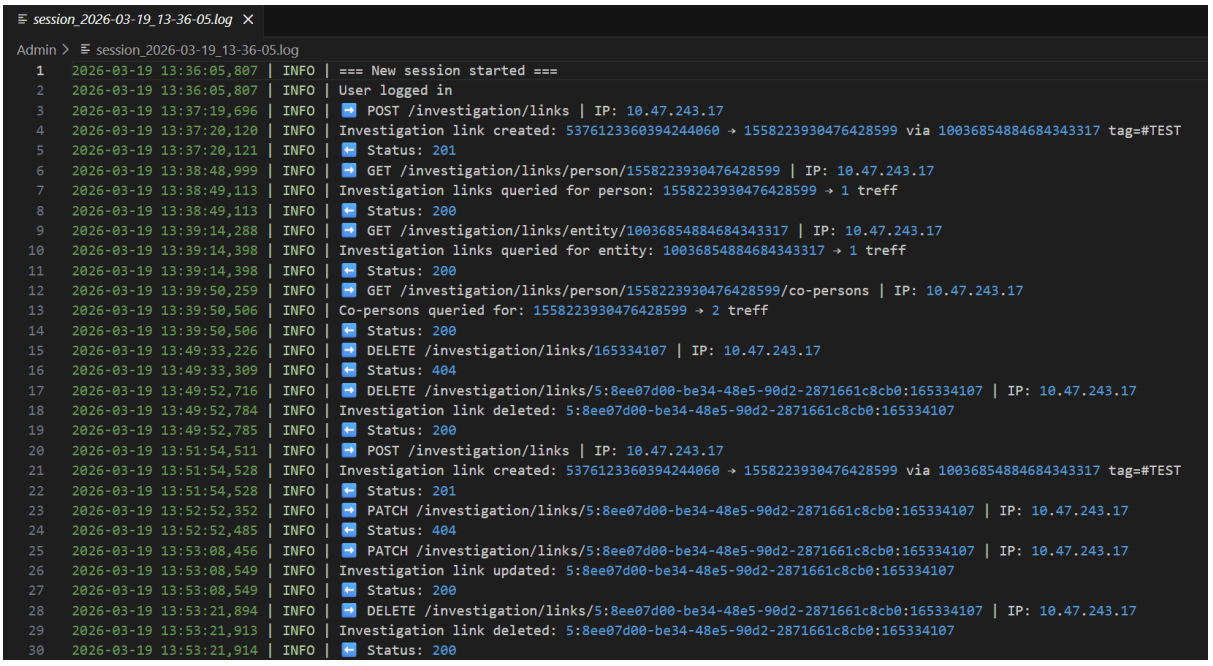
Figur 17: Authorized

API-et logger alle forespørsler fra hver bruker til en lokal fil på serveren, denne funksjonen bruker også tokens for å logge forespørslene til riktig bruker figur (18) viser loggene for brukeren Admin og test.



Figur 18: API-logger

Et eksempel fra loggen fra en sesjon til Admin brukeren med dato, tidspunkt, requests, koder og lignende kan du se under figur (19)



Figur 19: Eksempel fra loggen til Admin-brukeren

4.6 Implementasjon av infrastruktur

I prosjektet har den virtuelle maskinen som er levert via tjenesten Pegasus vært helt nødvendig. Grunnen til dette er siden at gruppens laptopper har altfor lite datakraft og lagringsplass for å parse og oppbevare hele datasettet lokalt i Neo4j-databasen. Den virtuelle maskinen har mye lagring, prosessorkraft og RAM for at parsingen av datasettet skulle gå effektivt og at spørringer mot databasen går kjapt. Det som kjøres på maskinen er Neo4j-databasen og FastAPI-et. Maskinen er satt opp i et Linux-basert miljø og bruker imaget til Ubuntu versjon 24.04. Den benytter flavoret som kalles aem.12c.64r.50g dette

flavoret gir den virtuelle maskinen 12 virtuelle CPU-kjerner, 64 gigabyte RAM og 300 GB lagringsplass. Denne datakraften har vist seg å være tilstrekkelig til prosjektets behov.

5 Prosessdokumentasjon

Dette kapittelet beskriver arbeidsprosessen i prosjektet, alt ifra planleggingen til den ferdige løsningen. Målet med kapittelet er å gi innsikt i hvordan utviklingen har gått, valg som er tatt underveis og hvordan utfordringene har blitt håndtert.

5.1 Metodisk tilnærming

I prosjektet ble arbeidet med utviklingen av kode gjennomført stegvis, der funksjonaliteten ble utviklet og forbedret underveis. Gruppen jobbet som regel separat med ulike deler av løsningen som besto av API-utvikling og parsing av noder og relasjoner.

For å sikre at det ble fremdrift i prosjektet ble det gjennomført ukentlige møter der statusen og eventuelle utfordringer ble diskutert. De forskjellige oppgavene ble fordelt mellom gruppemedlemmene for at flere deler av systemet kunne utvikles separat.

De to kodene for parsing av datasettet (noder og relasjoner) ble kjørt og endret flere ganger helt til at de riktige nodene og relasjonene kom inn i Neo4j-databasen på en effektiv måte. API-endepunktene ble også testet fortløpende for å verifisere funksjonalitet og eventuelle feil ble identifisert og rettet underveis.

5.1.1 Utviklingsmetode

I prosjektet var arbeidsformen en slags uformell smidig utviklingsmetode. Det ble ikke brukt noen spesifikk metode som for eksempel Scrum eller Kanban, men prosessen hadde flere likhetstrekk med smidig utvikling.

Vi jobbet iterativt med utviklingen av kodene, der funksjonaliteten ble utviklet i mindre steg og forbedret underveis. Prosjektet besto av flere iterasjoner der funksjonaliteten ble forbedret. Oppgavene ble håndtert fortløpende, noe som kan sammenlignes med en enkel backlog.

De ukentlige møtene ble brukt til å fordele oppgaver, sjekke fremdriften, og diskutere eventuelle utfordringer som er tilsvarende Scrum standups.

Tilnærmingen som ble valgt gjorde det mulig for gruppen å jobbe fleksibelt og tilpasse løsningen i løpet av prosjektet.

5.1.2 Teknisk metode

Tekniske metoder i prosjektet er sentrert rundt datahåndtering, modellering og API-utvikling. For å sikre effektiv behandling av de store datamengdene fra OpenOwnership datasett; *Register*, ble det benyttet en kombinasjon av programmeringsspråkene Python og Rust.

Valg av database: Neo4j

Som primær database valgte gruppen Neo4j. Dette valget støttes av teorien om at grafdatabaser er overlegne relasjonsdatabaser der man skal analysere dype koblinger og komplekse, indirekte eierskap i store nettverk (Neo4j, u.å.-d). Grafmodellen tillater effektiv traversering av relasjoner uten de ytelsesmessige kostnadene som ofte oppstår med komplekse sammenføyninger (joins) i SQL-databaser.

Backend og API: FastAPI

For å eksponere dataene ble FastAPI benyttet, et moderne web-rammeverk for Python. Ifølge den offisielle dokumentasjonen er FastAPI utviklet for å maksimere ytelse gjennom asynkron programmering (FastAPI, u.å.-a). En sentral fordel med valget er den innbygde datavalideringen via Pydantic-modeller, som sikrer at løsningen er både rask og robust mot feilaktig input fra FastAPI (FastAPI, u.å.-a).

Hybridløsning med Rust og Python

Python ble valgt som hovedspråk for parsing og API-utvikling i samråd med oppdragsgiver, da dette er politiets foretrukne språk og tilbyr et omfattende økosystem av biblioteker for datavalidering. Samtidig er Python anerkjent for sin enkelhet i utviklingsfasen, mens Rust utfyller dette ved å tilby overlegen ytelse i beregningstunge operasjoner (Mishra, 2023). Det ble tidlig identifisert at Pythons standarddrivene for Neo4j ikke var optimale for massiv datainnlastning av millioner av rader.

For å løse dette, ble det implementert en teknisk metode basert på en Rust-basert Neo4j-driver som kommuniserer via Bolt-protokollen. Valget av Rust er begrunnet i språkets evne til å håndtere minneffektivitet og parallelliseringskapasiteter uten risiko for minnefeil (PRG, u.å.). Ved å flytte den kritiske importlogikken til Rust, kunne gruppen oppnå en importytelse som teoretisk sett er betydelig raskere enn tradisjonelle Python-metoder. Denne hybride tilnærmingen gjorde det mulig å transformere flate JSON-data fra OpenOwnership til en kompleks grafmodell i Neo4j på en tidseffektiv måte.

Arkitekturvalg: API → Database

Systemarkitekturen følger et lagdelt mønster. Ved å skille API-logikken fra databaseinteraksjonene oppstår vi en løs kobling *(loose coupling)*. Forespørsler fra klienten mottas av FastAPI, som validerer input før den sender en Cypher-spørring via en driver til Neo4j. For å maksimere ytelsen under importfasen ble det, som beskrevet i (Bauer, 2024), en Rust-basert driver som fungerer som en bro for å overvinne Pythons GIL-begrensninger (Global Interpreter Lock).

Kodeprinsipper og Clean Code

Prosjektet har fulgt prinsippene for Clean Code og modulær struktur. Koden er delt inn i logiske moduler:

- main.py: Starter applikasjonen.

- schemas.py: Definerer datastrukturer.
- crud.py: Inneholder all database-logikk (Create, Read, Update, Delete).
- parsing/: Egne skript for å transformere rå JSON-data til graf-format. Denne modulariteten gjør at teamet kan jobbe på ulike deler av systemet samtidig uten konflikter, og det forenkler fremtidig vedlikehold og videreutvikling hos PIT.

5.1.3 Testmetode

Testing er en kritisk komponent for å sikre at politiets analytikere kan stole på dataene systemet presenterer. Testmetoden er delt inn i tre nivåer; integrasjonstesting, ytelsestesting og integritetskontroll.

Integrasjonstesting av database (test.py)

Src > Test.py > ...

```python
from neo4j import GraphDatabase

# URI examples: "neo4j://localhost", "neo4j+s://xxx.databases.neo4j.io"
URI = "neo4j://127.0.0.1:7687"
AUTH = ("neo4j", "gw3!PkG2uUQJ4bQ")

with GraphDatabase.driver(URI, auth=AUTH) as driver:
    driver.verify_connectivity()
    print("Connection established.")


#Kode fra ChatGPT - for å sjekke om koblingen ble laget.
driver = GraphDatabase.driver(URI, auth=AUTH)

with driver.session() as session:
    print("Connection established.")

    # Test-spørring
    result = session.run("MATCH (p:Person) RETURN p.name AS name")

    for record in result:
        print("Fant person:", record["name"])

driver.close()
```

Figur 20: Test.py

Det ble utviklet et eget skript, *test.py,* for å verifisere at applikasjonslaget kan kommunisere korrekt med Neo4j-databasen. Dette bekrefter at brukernavn, passord og nettverkstilkoblingen til Pegasus-serveren er konfigurert korrekt før ressurskrevende

operasjoner settes i gang. Som vist i figur 20, utfører skriptet en enkel spørring for å bekrefte at databasen returnerer forventede verdier.

Ytelsestesting og «Sweet Spot»

Siden systemet håndterer over 116 millioner noder, gjennomførte vi omfattende ytelsestesting av importprosessen. Vi testet ulike «batch-størrelser» (antall noder som sendes per transaksjon) for å finne den mest effektive metoden.

- **Optimal batch-størrelse:** Testene viste at **7000 noder per batch** var det mest optimale («Sweet Spot») for vår infrastruktur.
- **Parallellisering:** Vi testet antall «arbeidere» (tråder) og fant at **12 arbeidere** utnytter maskinvaren (12 kjerner og 64 GB RAM) best uten at systemet ble overbelastet. Dette reduserte importtiden betraktelig og sikret stabil drift gjennom hele prosessen.

3. Validering og integritetskontroll

For å sikre dataintegritet og forhindre tap av informasjon, ble det implementert en innebygd tellemekanisme i importskriptet. Når en import er fullført, genererer skriptet en rapport i terminalen som gir full oversikt over:

- Nøyaktig antall opprettede noder spesifisert per type (f.eks. PersonStatement, EntityStatement).
- Antall relasjoner og deres respektive typer.

Disse tallene ble kontrollert mot mindre test-datasett (som 1000.json) for å bekrefte at transformasjonslogikken var 100% korrekt. Denne metodikken ga prosjektgruppen trygghet på at logikken var robust før den ble skalert opp til det fulle datasettet.

Testing er avgjørende i dette prosjektet fordi systemet skal håndtere informasjon som kan være kritisk for politiets etterretningsarbeid. Ufullstendige data eller feilaktige koblinger i

grafen kan føre til feilanalyser. Derfor prioriterte gruppen grundig verifisering av både dataintegritet og systemytelse for å sikre en pålitelig og robust løsning.

5.1.4 Dokumentasjon og kommunikasjon

Gjennomføring av prosjektet har krevd en strukturert tilnærming til dokumentasjon og informasjonsflyt, både internt i gruppen og eksternt mot oppdragsgiver.

Intern kommunikasjon og prosjektstyring

For å sikre en effektiv fremdrift har gruppen benyttet seg av faste rutiner for samhandling. Dette inkluderer:

1. **Ukentlige møter:** Gruppen har avholdt faste møter for å synkronisere arbeidsoppgaver, diskutere tekniske utfordringer og ta beslutninger underveis. Alle møtenotatene kan sees i Vedlegg 2: Møtenotater

2. **Arbeids- og milepælsplan:** Det ble tidlig utarbeidet en tydelig milepælsplan som har fungert som et styringsverktøy gjennom hele prosjektet. Ved å bryte ned prosjektet i mindre, målbare leveranser, har gruppen opprettholdt en forutsigbar arbeidsflyt.

3. **Redusert miskommunikasjon:** Denne strukturerte tilnærmingen har vært avgjørende for å minimere misforståelser og sikre at alle gruppemedlemmer til enhver tid har hatt innsikt i prosjektets helhetlige status.

Teknisk dokumentasjon og kildekode

All kildekode er lagret i et felles repository på GitHub, som fungerer som gruppens primære verktøy for versjonskontroll. Dokumentasjonen følger prinsippet om «dokumentasjon som kode», der API-spesifikasjonene genereres automatisk via OpenAPI-standarden i FastAPI. Dette sikrer at dokumentasjonen alltid er i samsvar med den faktiske implementasjonen. For å opprettholde høy kodekvalitet ble det benyttet p*ull* requests, der gruppemedlemmer går gjennom hverandres endringer før de integreres i hovedprosjektet.

Kommunikasjon med oppdragsgiver

Samarbeidet med Politiets IT-enhet (PIT) har vært preget av en smidig tilnærming og tett dialog. For å sikre profesjonalitet og sporbarhet i prosjektet, har gruppen:

- **Gjennomført regelmessige statusmøter:** Dette har sikret at utviklingen hele tiden har vært i tråd med politiets behov for verktøy som kan avdekke økonomisk kriminalitet.
- **Utarbeidet møtereferater:** Det er ført referater fra samtlige møter med oppdragsgiver. Disse fungerer som formell dokumentasjon på beslutninger, kravendringer og avklaringer som er gjort underveis. Dette har vært et viktig verktøy for å sikre at både gruppen og PIT har hatt en felles forståelse av prosjektets retning.

Den strukturerte tilnærmingen til både intern og ekstern kommunikasjon har vært en avgjørende suksessfaktor for prosjektet. Kombinasjonen av faste møtepunkter, detaljerte referater og moderne utviklingsverktøy som GitHub og automatiserte API-spesifikasjoner, har skapt en transparent arbeidsprosess. Dette har minimert risikoen for misforståelser, men også at alle tekniske valg er godt forankret i oppdragsgiverens behov. Resultatet er en veldokumentert løsning der kildekode, metodikk og beslutningsgrunnlag er sømløst integrert, noe som legger til rette for en effektiv overlevering og fremtidig videreutvikling hos Politiets IT-enhet.

5.1.5 Prosjektfaser

Prosjektet ble gjennomført i henhold til en strukturert milepælsplan som sporet progresjon fra de første teoretiske modellene til en ferdig API med validert data. Arbeidet kan deles inn i fire hovedfaser:

Fase 1: Planlegging og modellering (Uke 6–8)

I den innledende fasen var fokuset på å definere systemets arkitektur og datamodell. Dette handlet om forbedring av ER-diagrammet og bestemmelse av de endelige nodene og

relasjonene i grafen. En sentral milepæl var å validere logikken mot testsettet 1000.json for å avdekke eventuelle logiske feil i transformasjonen til BODS-standarden. En tett dialog med klienten ble også etablert for å få godkjent modelleringsmetoden.

Fase 2: Teknisk import og ytelsesoptimalisering (Uke 9–10)

Etter godkjent modell startet arbeidet med å parse og importere hele datasettet på over 116 millioner noder og 165 millioner relasjoner til Neo4j-databasen. Det var i denne fasen gruppen gjennomførte omfattende ytelsestester for å finne systemets «Sweet Spot». Ved å eksperimentere med ulike batch-størrelser og antall **arbeidere,** ble det fastslått at en **batch på 7000** og **12 parallelle arbeidere** ga den mest effektive utnyttelsen av maskinvaren for nodeimporten. For importen av relasjonene forble batch størrelsen uendret, men antall arbeidere ble endret fra 12 til 6 siden relasjonsimporten er mer omfattende.

Fase 3: Utvikling av API og funksjonalitet (Uke 11–13)

Da datagrunnlaget var på plass og validert, startet utviklingen av API-et i FastAPI. Milepælene i denne fasen inkluderte blant annet:

- Etablering av kobling mellom Neo4j og API-et.
- Implementering av funksjonalitet for innlogging og sesjonsbasert logging i Swagger.
- Utvikling av endepunkter for å hente detaljert informasjon om personer og eierskap.

Fase 4: Analyse, testing, drøfting og ferdigstilling (Uke 15–23)

Den siste fasen fokuserte på å sammenstille «kvalitetstall» og verifisere om de tekniske funnene svarte på problemstillingen. Dette inkluderte Cypher-analyser for å demonstrere eierkjeder og mønstergjenkjenning i det store datasettet. Prosjektet ble avsluttet med utarbeidelse av sluttrapport, plakat og forberedelser til den endelige presentasjonen 2. juni.

6 Implementasjon / Produktdokumentasjon

Dette kapittelet beskriver hvordan implementasjonen av løsningen har blitt gjennomført og de viktigste komponentene i systemet. Det beskrives hvordan dataene fra datasettet OpenOwnership er behandlet, lagret og gjort tilgjengelig gjennom et API.

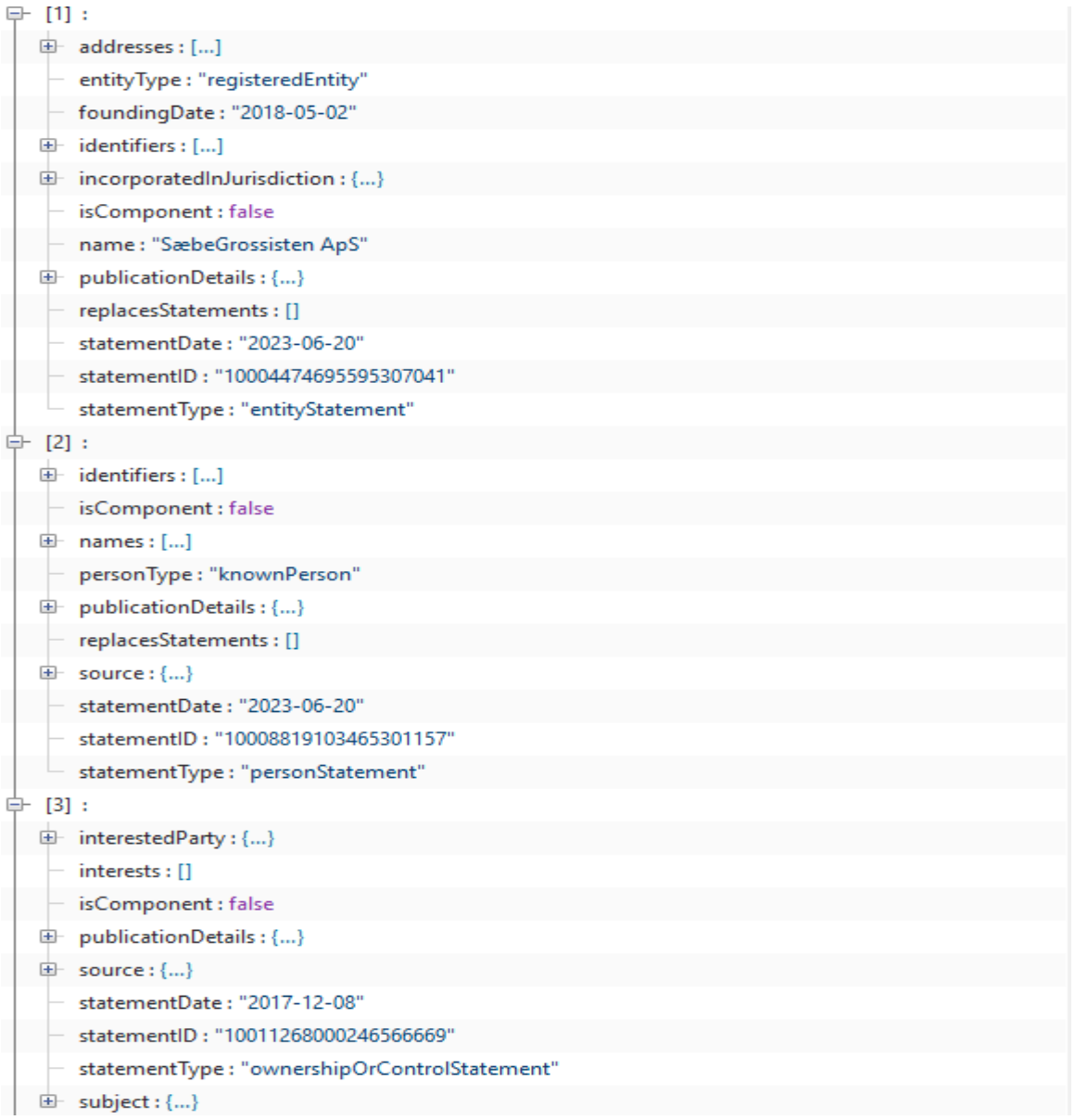
6.1 Datainnhenting og parsing

6.1.1 Datasettbeskrivelse

I dette prosjektet ble Open Ownership Register datasettet benyttet. Dette datasettet inneholder eierdata for reelle eierskap fra registerene fra Danmark, Slovakia og Storbritannia. Datasettet var opprinnelig lastet ned fra OpenOwnership sitt BODS-register i JSON-format (OpenOwnership, 2026). Datasettet har nå dessverre blitt fjernet fra originalkilden, men man kan få tilgang til en arkivert versjon av siden via nettsiden Wayback Machine for referanse (Internet Archive, 2026). Lenken til datasettet i WaybackMachine ligger under.

https://web.archive.org/web/20251127040344/https://bods-data.openownership.org/source/register/

Størrelsen til datasettet er på rundt 38 gigabyte og inneholder 36.508.905 linjer hvor hver linje er et objekt som beskriver enten en person (*PersonStatement*), Selskap (*EntityStatement*) eller en økonomisk relasjon mellom selskap og/eller personer (*OOCStatement*). Datastrukturen er organisert som JSON-objekter med arrays og nested objects, disse inneholder metadata og identifikatorer som er knyttet til hver entitet. Et utdrag fra JSON-filen til databasen vist i Dadroit JSON Viewer kan ses under figur (21)

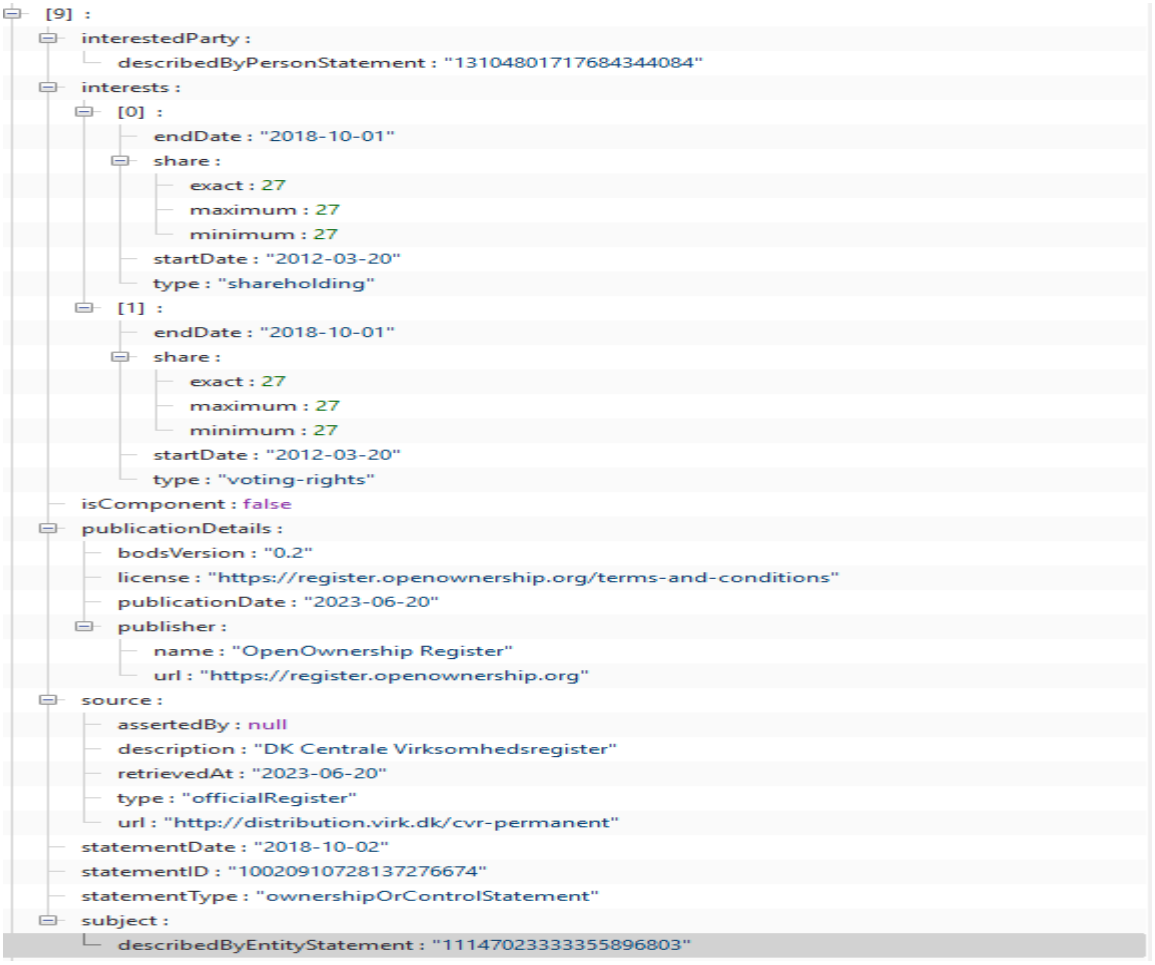


Figur 21: Skjermbilde fra Dadroit JSON Viewer
Kilde: https://dadroit.com/

Datasettet brukes til å avdekke økonomisk kriminalitet ved at etterforskere ser på eierskap og økonomiske forbindelser mellom personer og firmaer.

6.1.2 Parsing og forbehandling

Datasettet fra OpenOwnership ble lastet inn som JSON-linjer og parset ved hjelp av orjson. Hver eneste linje i filen representerer et objekt som ble konvertert fra tekst til en Python-datastruktur. Figur (22) viser et ownershipOrControlStatement som inneholder veldig viktig informasjon om eierskap og kontroll. Dette er en viktig del av datastrukturen som må behandles korrekt under parsing.



Figur 22: Skjermbilde av et statement i Dadroit JSON Viewer
Kilde: https://dadroit.com/

Det er to koder, en for import av noder og en for import av relasjoner, først kjøres koden for nodeimport og deretter koden for relasjonsimporten. Begge kodene er veldig like, figur 23 viser hvordan dataen blir innlest

```
try:
    with open(file_path, "rb") as f:
        f.seek(start_byte)
        if start_byte > 0:
            f.readline()

        while True:
            if f.tell() > end_byte:
                break

            line = f.readline()
            if not line: break

            try:
                obj = orjson.loads(line)
            except:
                continue

            stype = obj.get("statementType")
            if not stype: continue
```

Figur 23: Innlesning av data

Koden åpner JSON-filen spesifisert i file_path og leser dataene linje for linje ved hjelp av en while-løkke. Hver eneste linje parsres til et objekt ved bruk av orjson.

Hvis objektet inneholder et gyldig statementType, blir informasjonen hentet ut og behandlet videre. Objekter uten statementType blir ignorert og løkken fortsetter til neste linje i datasettet.

Etter at dataene har blitt lest inn og parset, blir objektene klassifisert basert på StatementType. Videre blir informasjonen ekstrahert fra hvert objekt, dette inkluderer både enkle felt og nested strukturer. Figur 24 viser et eksempel på hvordan dataen blir ekstrahert fra personStatement:

```
if stype == "personStatement":

    batches["personStatement"].append({
        "statementID": sid,
        "statementType": obj.get("statementType"),
        "isComponent": obj.get("isComponent"),
        "replacesStatements": obj.get("replacesStatements"),
        "statementDate": obj.get("statementDate"),
        "publicationDetails_bodsVersion": obj.get("publicationDetails", {}).get("bodsVersion"),
```

Figur 24: personStatement

Her blir dataene transformert og organisert i egne datastrukturer, og deretter blir de samlet i batcher for videre behandling. publicationDetails_bodsVersion er et eksempel på en nested struktur siden koden må først gå inn i publicationDetails og deretter hente bodsVersion.

Både *PersonStatement, EntityStatement* og *OOCStatement* har flere batcher innenfor seg selv i koden, en batch for hver av de 9 nodene i databasedesignet.

6.2 Import til Neo4j

6.2.1 Datamodell (noder)

I Neo4j er datastrukturen modellert som en graf, der informasjonen er representert av ulike nodetyper. Hver nodetype tilsvarer en sentral entitet fra datasettet.

Nodetypene i modellen og forklaring av dem vises i tabellen under

Nodetype	Beskrivelse
Source	Kilder til informasjonen med url og publiseringsdato
PersonStatement	Representerer en registrering av en person, inkludert metadata om publisering, id og lignende
EntityStatement	Representerer en registrering av et firma inkludert metadata om publisering, id og

	lignende
Identifier	Representerer unike identifikatorer med unike ider
Person	Representerer en enkeltperson med grunnleggende identitetsinformasjon som navn, fødselsdato, familienavn og lignende
Entity	Representerer et firma med grunnleggende informasjon om firmanavn, lokasjon, grunnlegningsdato og lignende
Nationality	Representerer nasjonalitet knyttet til en person
Address	Representerer adresser knyttet til personer eller firmaer
OOCStatement	Representerer eierskap- eller kontrollforhold mellom aktører i datasettet

Disse nodene er basert på strukturen i JSON-datasettet, der hvert eneste objekt blir transformert til en eller flere noder i grafdatabasen.

6.2.2 Relasjoner

De forskjellige nodetypene i datamodellen blir koblet sammen med relasjoner i grafdatabasen. Disse relasjonene beskriver forbindelsene mellom personer, selskaper og tilknyttet informasjon. Disse relasjonene er nødvendige for å kunne representere komplekse eierskapsstrukturer i datasettet. I tabellen under er det en forklaring for hver relasjon og hvordan de er koblet sammen.

Relasjon	Beskrivelse

Has_Statement	Kobler Source til Personstatement, EntityStatement, OOCStatement
Has_Identifier	Kobler PersonStatements og EntityStatements til identifier-noder
Describes_Person	Kobler en PersonStatement til Person og viser hvilken person registreringen gjelder
Describes_Entity	Kobler en EntityStatement til Entity og viser hvilken Entity registreringen gjelder
Has_Nationality	Kobler en Person til Nationality
Has_address	Kobler en Person eller Entity til en Address
Is_Interested_Party	Kobler en Person eller Entity som har kontroll eller eierskap i en annen Entity til en OOCStatement
Descibes_Control	Kobler en OOCStatement til en Entity og beskriver kontroll
Describes_Ownership	Kobler en OOCStatement til en Entity og inneholder aksjeinformasjon på relasjonen

Relasjonene mellom nodene kobles ved bruk av unike identifikatorer som p*ersonID, entityID* og *statementID*. Disse blir brukt i importprosessen for å matche og sikre en korrekt kobling mellom nodene i Neo4j.

6.2.3 Cypher og importlogikk

Importen til Neo4j skjer ved hjelp av Cypher-spørringer. Disse brukes til å opprette og oppdatere noder og relasjoner i grafdatabasen basert på de strukturerte batchene fra forbehandlingen av dataene.

I koden er det nesten utelukkende brukt Merge for å unngå duplikater, bortsett fra opprettelse av relasjonene *Describes_Ownership*. Grunnen til at denne ene relasjonen måtte være på CREATE var siden at 1 million viktige relasjoner forsvant ved bruk av MERGE. I datasettet er det flere eierskapsrelasjoner mellom OOCStatements og Entitys og med merge blir bare den første mellom samme noder opprettet og de andre ignorert som gir mye tap av viktig data.

Figur 25 under viser et eksempel på Cypher-spørringene for import av personnodene

```
def import_persons(tx, batch):
    tx.run("""
        UNWIND $batch AS obj
        MERGE (p:Person {personID: obj.personID})
        SET
            p.fullName      = COALESCE(obj.fullName, p.fullName),
            p.givenName     = COALESCE(obj.givenName, p.givenName),
            p.familyName    = COALESCE(obj.familyName, p.familyName),
            p.birthDate     = COALESCE(obj.birthDate, p.birthDate),
            p.type          = COALESCE(obj.type, p.type),
            p.personType    = COALESCE(obj.personType, p.personType),
            p.CountryCode   = COALESCE(obj.Countrycode, p.CountryCode),
            p.Country       = COALESCE(obj.Country, p.Country)
    """, batch=batch)
```

Figur 25: Cypher-Spørringer for Person nodene

Hver batch behandles ved hjelp av UNWIND som gjør det mulig å prosessere flere objekter i en spørring. Man kan se at det merges på den unike personIDen og da sjekkes det i databasen om det allerede er en node med den personIDen eller ikke. Hvis PersonIDen ikke fins fra før opprettes noden, men hvis den fins fra før opprettes ikke noden. Alt av informasjon noden skal inneholde blir satt med Cypher sitt SET-klausul.

6.2.4 Batcher og Ytelse

Objekter av samme type ble samlet i batcher, grunnen til dette var for å redusere antall operasjoner mot databasen. Dette forbedrer ytelsen og gir større kontroll ved håndtering av

store datamengder. Når batchene nådde en viss størrelse som var definert øverst i koden, ble de sendt videre for lagring i databasen.

6.2.5 Importprosessen

Før importprosessen kunne starte måtte noen innstillinger i Neo4j sin config-fil endres for å forbedre ytelsen og stabiliteten. I figurene under kan man se hva slags innstillinger gruppen brukte i Neo4j sin config-fil..

```
#*****************************************************************
# Network connector configuration
#*****************************************************************

# With default configuration Neo4j only accepts local connections.
# To accept non-local connections, uncomment this line:
server.default_listen_address=0.0.0.0
```

Figur 26: Nettverks konfigurasjonen til Neo4

Disse innstillingene gjør det mulig for alle på OsloMet sitt internett å åpne gruppens Neo4j-browser. Databasen er beskyttet bak passord som gjør at det ikke er et sikkerhetsproblem. Denne innstillingen ble satt for å gjøre det lettest mulig for gruppen å aksessere databasen figur(26).

```
# Bolt connector
server.bolt.enabled=true
server.bolt.tls_level=DISABLED
server.bolt.listen_address=:7687
server.bolt.advertised_address=10.196.241.255:7687
```

Figur 27: Bolt tilkobling innstillingene til Neo4j

Disse innstillingene er for at det skal være mulig å koble til Neo4j via bolt-protokollen for å parse inn nodene og relasjonene med import-kodene figur(27).

```
# HTTP Connector. There can be zero or one HTTP connectors.
server.http.enabled=true
server.http.listen_address=:7474
server.http.advertised_address=10.196.241.255:7474
```

Figur 28: HTTP-tilkobling innstillingene til Neo4j

Disse innstillingene figur(28) er satt for at det skal være mulig å koble til Neo4j-Browser

```
# Java Heap Size: by default the Java heap size is dynamically calculated based
# on available system resources. Uncomment these lines to set specific initial
# and maximum heap size.
server.memory.heap.initial_size=16G
server.memory.heap.max_size=16G

# The amount of memory to use for mapping the store files.
# The default page cache memory assumes the machine is dedicated to running
# Neo4j, and is heuristically set to 50% of RAM minus the Java heap size.
server.memory.pagecache.size=40G
```

Figur 29: Minneinnstillingene til Neo4j

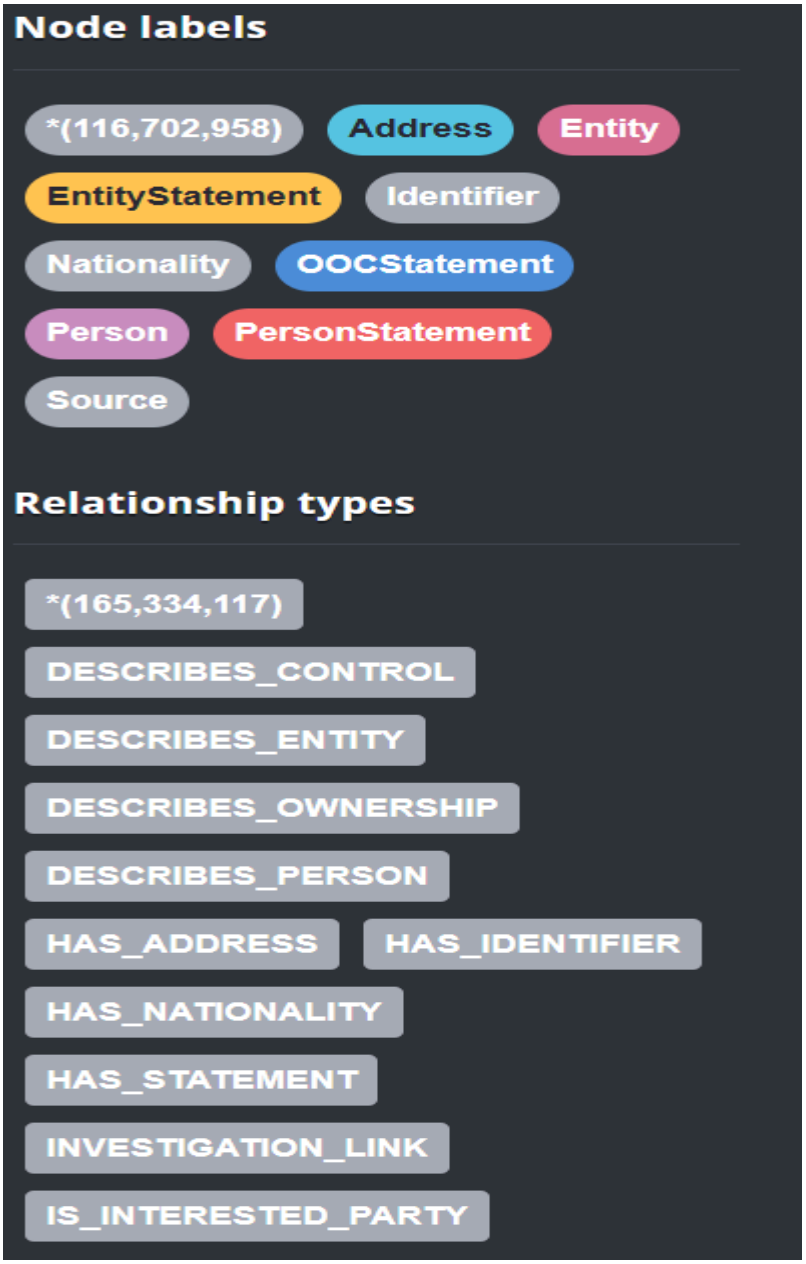
Disse minneinnstillingene figur (29) ble satt for å utnytte de 64 GB med RAM som VMen hadde på en effektiv måte. Det ble satt 16 GB til Heap maks og 40 GB til page cache som gir en total maks minnebruk for Neo4j på 56 GB. Disse innstillingene sørger for at det blir 8 GB minne til overs til operativsystemet som er viktig for stabiliteten.

Først ble nodeimporten kjørt, der nodene ble opprettet i Neo4j. Etter at alle nodene var importert inn i databasen, ble relasjonsimporten kjørt for å koble sammen de ulike entitetene. Hele importprosessen er beskrevet i tabellen under

Steg	Beskrivelse
1. Filinnlesning	JSON-filen leses linje for linje fra disk
2. Parsing	orjson benyttes for å parse hver linje til et

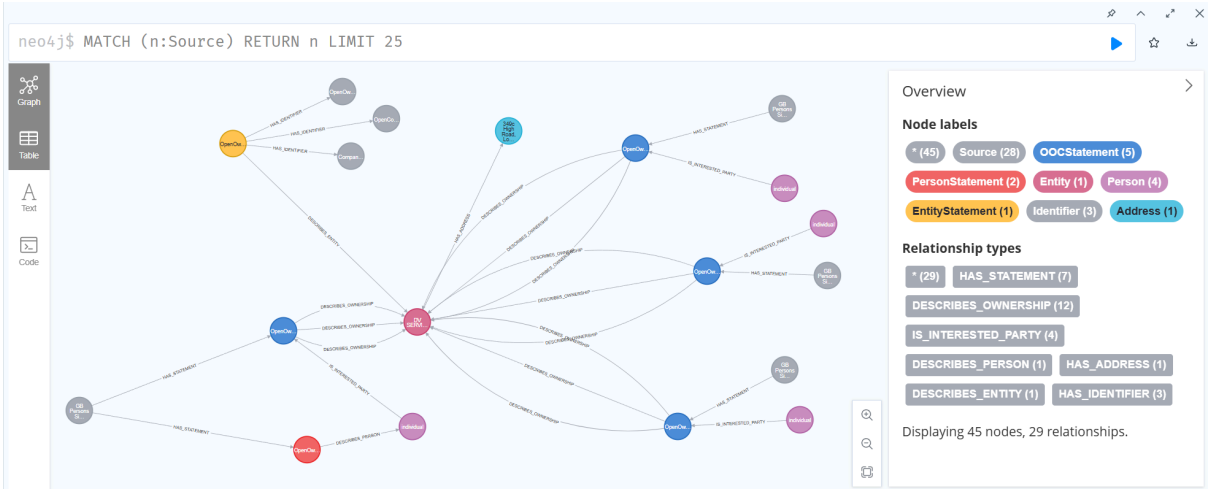
	JSON-objekt
3. Klassifisering	Objektene sorteres etter statementType (Person, Entity, OOC)
4. Batching	Dataene samles i batcher for å redusere antall databaseoperasjoner
5. Parallelisering	Flere prosesser(arbeidere) brukes for å utnytte flere CPU-kjerner ved at JSON-filen deles opp mellom arbeiderne
6. Nodeimport	Nodene blir opprettet først i Neo4j
7. Relasjonsimport	Relasjonene kobles til de eksisterende nodene etter opprettelse
8. Ferdigstilling	Dataen lagres og systemet verifiseres ved bruk av en tellekontroller

Figur (30) viser antallet av noder og relasjoner etter ferdig import til Neo4j-databasen, det er cirka 116,7 millioner noder og 165,3 millioner relasjoner



Figur 30: Noder or Relasjoner

Figur (31) viser et eksempel på hvordan noen av nodene og relasjonene er koblet sammen.



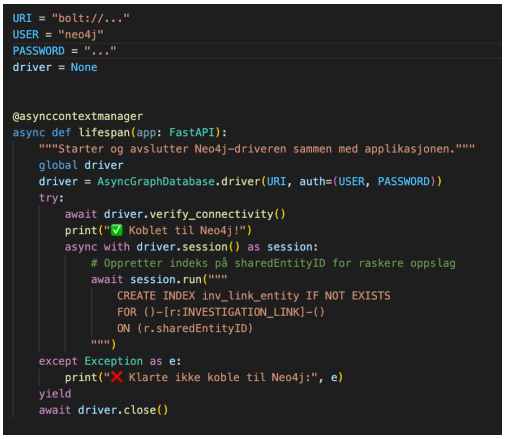
Figur 31: Eksempel på noder og koblinger i Neo4j

6.3 API-utvikling (FastAPI)

I dette prosjektet ble det utviklet REST API med hjelp av FastAPI. API-et jobber som mellomlaget mellom klienten og Neo4j. Dette gjør det mulig å hente ut informasjon om personer, selskaper og eierskap, uten at klienten må kontakte databasen direkte.

API-et kobler til databasen med *AsyncGraphDatabase.driver*. Denne tilkoblingen håndteres i en *lifespan*-funksjon, hvor den sørger for at Neo4j-driveren åpnes og lukkes ut ifra applikasjonen. Samtidig åpnes det en indeks på *sharedEntity* for relasjonen *INVESTIGATION_LINK,* som gjør oppslag på etterforskningsrelajoner raskere.

Figur (32) viser Neo4j kolbing til API:



```
URI = "bolt://..."
USER = "neo4j"
PASSWORD = "..."
driver = None


@asynccontextmanager
async def lifespan(app: FastAPI):
    """Starter og avslutter Neo4j-driveren sammen med applikasjonen."""
    global driver
    driver = AsyncGraphDatabase.driver(URI, auth=(USER, PASSWORD))
    try:
        await driver.verify_connectivity()
        print("✅ Koblet til Neo4j!")
        async with driver.session() as session:
            # Oppretter indeks på sharedEntityID for raskere oppslag
            await session.run("""
                CREATE INDEX inv_link_entity IF NOT EXISTS
                FOR ()-[r:INVESTIGATION_LINK]-()
                ON (r.sharedEntityID)
            """)
    except Exception as e:
        print("❌ Klarte ikke koble til Neo4j:", e)
    yield
    await driver.close()
```

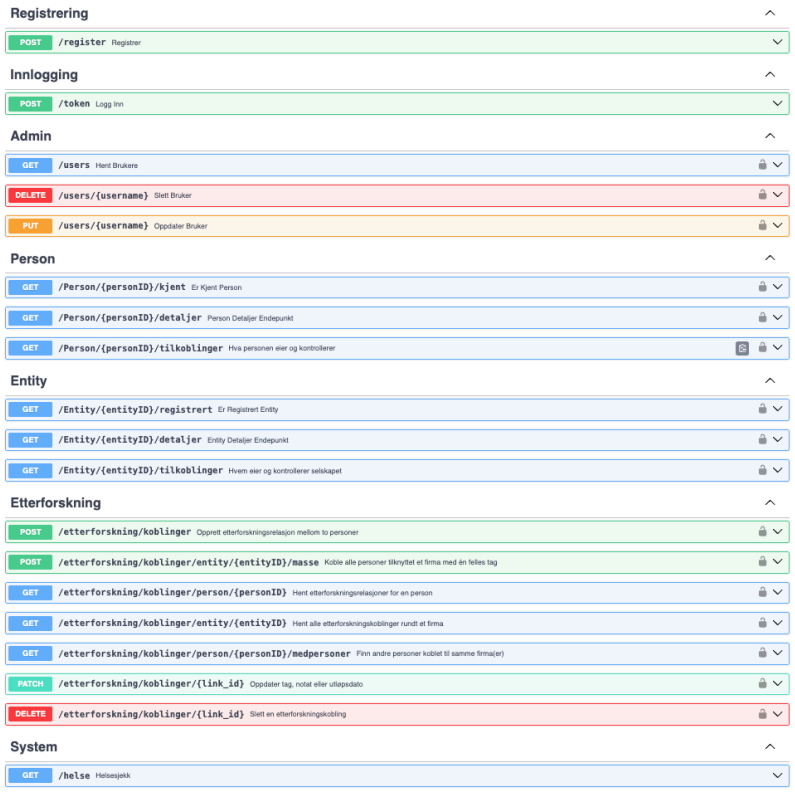
Figur 32: Neo4j-tilkobling

Endepunkt	**Beskrivelse**
POST /register	Registrerer en ny bruker
POST /token	Logger inn bruker og returnerer JWT-token
GET /users	Lister registrerte brukere*
DELETE /users/{username}	Sletter en bruker*
PUT /users/{username}	Oppdaterer passord for en bruker*
GET /Person/{personID}/kjent	Sjekker om en person finnes i grafen
GET /Person/{personID}/detaljer	Henter detaljinformasjon om en person
GET /Entity/{entityID}/registrert	Sjekker om et selskap finnes i grafen
GET /Entity/{entityID}/detaljer	Henter detaljinformasjon om et selskap
GET /Person/{personID}/tilkoblinger	Henter selskaper personen eier eller kontrollerer

GET /Entity/{entityID}/tilkoblinger	Henter eiere og kontrollører for et selskap
POST /etterforskning/koblinger	Oppretter en etterforskningskobling mellom to personer
POST /etterforskning/koblinger/entity/{entityID}/masse	Oppretter etterforskningskoblinger mellom alle personer knyttet til et selskap
GET /etterforskning/koblinger/person/{personID}	Henter etterforskningskoblinger for en person
GET /etterforskning/koblinger/entity/{entityID}	Henter etterforskningskoblinger knyttet til et selskap
GET /etterforskning/koblinger/person/{personID}/medpersoner	Finner andre personer koblet til samme firma
PATCH /etterforskning/koblinger/{link_id}	Oppdaterer en etterforskningskobling
DELETE /etterforskning/koblinger/{link_id}	Sletter en etterforskningskobling
GET /helse	Sjekker om API-et og databasen er tilgjengelige

* = kun for admin

Skjermbilde figur (33) som viser swagger dokumentasjon som genereres automatisk av FastAPI



Figur 33: API-endepunkter

6.3.1 Responsmodeller og validering

API-et bruker Pydantic-modeller. Dette er for å strukturere dataen som sendes inn og returneres fra API-et. Dette gjør at API-et automatisk validerer input og gir responsen en fast struktur.

Modellen *PersonDetaljerSvar* brukes når API-et returnerer informasjon om en person. Denne inneholder felt som *personID, fullName, birthDate, familyName, givenName, type* og *personType.* Selskap, eierskap, kontrollforhold og etterforskningskoblinger bruker like modeller for ryddigere og enklere bruk for klienter.

6.3.2 Kommunikasjon med Neo4j

Kommunikasjonen mellom Neo4J og API-et skjer gjennom asynkrone Cypher-spørringer. Endepunktene bruker funksjonen *hent_okt(),* som henter en database økt og gir endepunktene tilgang til en aktiv Neo4j-sesjon.

Funksjonen *hent_person_detaljer()* henter informasjon om en person ved å matche på *PersonID:* Figur (34)

```python
async def hent_person_detaljer(session, personID: str):
    """Henter alle detaljfelt for en Person-node."""
    sporring = """
    MATCH (p:Person {personID: $personID})
    RETURN p.personID AS personID,
           p.fullName AS fullName,
           p.birthDate AS birthDate,
           p.familyName AS familyName,
           p.givenName AS givenName,
           p.type AS type,
           p.personType AS personType
    """
    resultat = await session.run(sporring, personID=personID)
    rad = await resultat.single()
    if not rad:
        return None
    return dict(rad)
```

Figur 34: Persondetaljer Funksjon

Figuren viser hvordan API-et bruker parametere, *$personID* for å minske sjansen for feil ved spørringer.

6.3.3 Eierskap og kontroll

For eierskap og kontroll brukes endepunktet *GET /Person/{personID}/tilkoblinger,* som returnerer hvilke selskaper personen eier eller kontrollerer. Dette gjøre ved å følge grafstrukturen:

- Person -[:IS_INTERESTED_PARTY]-> OOCStatement -[:DESCRIBES_OWNERSHIP]-> Entity

- Person -[:IS_INTERESTED_PARTY]-> OOCStatement -[:DESCRIBES_CONTROL]-> Entity

Selskaper bruker funksjonen *GET /Person/{entityID}/tilkoblinger,* som returnerer personer eller andre selskaper som eier den selskapet.

6.3.4 Etterforskningsrelasjoner

API-et har funksjon for opprettelse av etterforskningskoblinger mellom personer, dette er med relasjonen INVESTIGATION_LINK i Neo4j. Denne relasjonen sjekker om to personer har noe til felles i en etterforskningskontekst. For eksempel at begge har eierskap i samme selskap. Relasjonen inneholder disse dataene:

Felt	**Beskrivelse**
sharedEntityID	Selskapet personene har til felles
tag	Etikett, for eksempel #hvitvasking eller #korrupsjon
note	Valgfritt notat
created_by	Brukeren som opprettet koblingen
created_at	Tidspunkt for opprettelse
expires_at	Eventuell utløpsdato

Før en kobling opprettes mellom to personer, validerer API-et at begge personene faktisk er koblet sammen gjennom OOC-grafen.

6.4 Klient / API-bruk

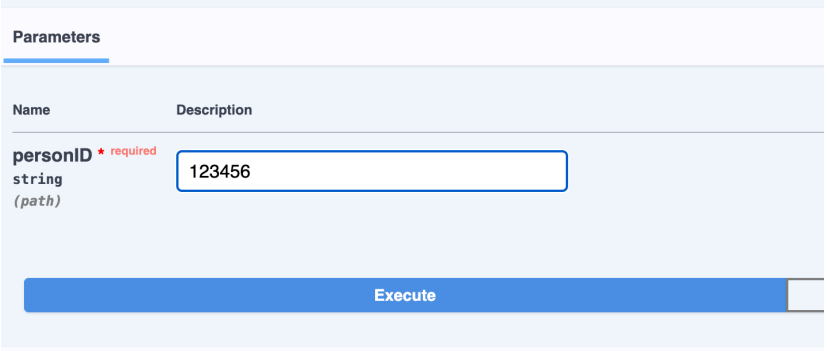
API-et er laget for bruk av ulike klienter. Det kan være en frontend-applikasjon, swagger eller egne HTTP-forespørsler. FastAPI genererer automatisk dokumentasjon gjennom swagger UI, som gjør det mulig å se endepunktene, sende forespørsler og se JSON-responsene.

6.4.1 Innlogging og token

Noen av endepunktene i API-et er beskyttet og krever innlogging. Brukeren må først sende brukernavn og passord til endepunktet *POST /token.* Dersom det går igjennom returnerer API-et en JWT-token, som sendes med i senere forespørsler i Authorization_headeren. Dette gjør at API-et vet hvilke bruker som har sendt forespørselen.

6.4.2 Eksempel på forespørsel

Under endepunktet *GET /Person/{personID}/detaljer* skriver man inn *personID*:



Figur 35: Uthenting av personopplysninger i APIet

Da får man ut en slik respons:

```
{
  "personID": "string",
  "fullName": "string",
  "birthDate": "string",
  "familyName": "string",
  "givenName": "string",
  "type": "string",
  "personType": "string"
}
```


Figur 36: Resultatet av uthentingen av personopplysninger

Ved søk av selskaper, skriver man inn *entityID,* og får opp andre variabler som *owns* og *controls.*

6.5 Logging og sikkerhet

Logging og sikkerhet er viktige deler i løsningen. Systemet inneholder autentisering, brukeradministrasjon, passordhashing, tilgangskontroll og logging av forespørsler.

6.5.1 Autentisering

For autentisering bruker API-et OAuth2 med Bearer-token og JWT. Ved vellykket innlogging generes det et signert JWT-token, som inneholder brukernavn under feltet *sub* og har en varighet på 60 minutter. Brukeren må være logget inn for å bruke beskyttede endepunkter (endepunkter under Admin, Person, Entity og Etterforskning), hvis ikke brukes returnerer API-et statuskode 401 ved forsøk.

6.5.2 Brukerdatabase og passord

Brukerne lagres lokalt i SQLite-databasen *users.db.* Tabellene består av brukernavn, hashet passord, admin-status og tidspunkt for opprettelse. Passordene hashes ved lagring med bcrypt gjennom passlib, dette gjøres for at det ikke skal være mulig å lese passordene fra databasen.

6.5.3 Tilgangskontroll

Systemet skiller brukerne mellom vanlige og admin-brukere, som gjøres i SQLite-databasen med feltet *is_admin*. Vanlige brukere har tilgang til søke- og etterforskningsfunksjonene, mens admin brukerne har tilgang til brukeradministrasjon, og endepunktene med DELETE og PUT.

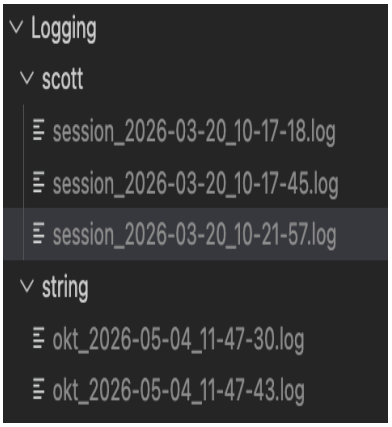
6.5.4 Logging

API-et har en egen logging-middelware, som logger alle forespørsler. Av forespørselen logges:

Loggdata	**Beskrivelse**
Brukernavn	Hentes fra JWT-token dersom brukeren er innlogget
HTTP-metode	For eksempel GET, POST, PATCH eller DELETE
Endepunkt	Hvilken URL-sti som ble brukt
IP-adresse	Klientens IP-adresse
Statuskode	Responsstatus fra API-et

Systemet logger også handlinger, som når en bruker logger inn, når etterforskningskobling opprettes, oppdateres eller slettes, og når masskoblinger opprettes.

Loggene lagres for hver bruker og hver økt i mappen Logging. Brukerne får hver sin undermappe og egen loggfil.



Figur 37: Eksempel på loggfilstruktur



Figur 38: Detaljert logg

Logging av forespørsler er veldig viktig for sikkerheten i systemet. Logging gjør det mulig å spore brukeraktivitet og undersøke om systemet har blitt misbrukt eller brukt av uautoriserte brukere. Logging gjør det også mulig å se hvilke handlinger som har blitt utført, for eksempel: søk, opprettelse, oppdatering eller sletting av data.

7 Testdokumentasjon

7.1 Teststrategi

Teststrategien i prosjektet har hatt som mål å sikre at systemet er korrekt, stabilt og ytelsesmessig tilstrekkelig for å støtte politiets analysearbeid. Siden feil i datagrunnlaget kan føre til feilaktige avgjørelser i etterforskning av økonomisk kriminalitet, ble det lagt stor vekt på dataintegritet og å verifisere at spørringer returnerte forventede resultater.

Testingen er delt inn i fem hovedkategorier: valideringstest, importkontroll, ytelsestest, aggregeringstest og skalerbarhetstest. Hver kategori tar for seg forskjellige aspekter av systemet, fra korrekthet i datastrukturen til responstid under belastning. Metodikken og verktøyene som ligger til grunn for testingen er beskrevet i kapittel 5.1.3. Dette kapittelet dokumenterer de spesifikke testene som ble utført og deres resultater.

7.2 Valideringstest

For å bekrefte at datastrukturen i Neo4j var korrekt etter import, ble et enkelt oppslag utført på kjente noder. Spørringen nedenfor ble brukt for å hente egenskapene til den første Person-noden i databasen:



Figur 39: Bilde av Cypher-spørring for valideringstest

Resultatet bekrefter at noden inneholder alle de forventet verdier som: familyName, givenName, fullName, personID, type, personType og birthDate, i samsvar med datamodellen definert i kapittel 4.2.

For å bekrefte at relasjoner og tilknyttede noder er korrekt koblet, ble det gjennomført et oppslag der PersonStatement peker mot Person via DESCRIBES_PERSON-relasjonen, i tråd med datamodellen:



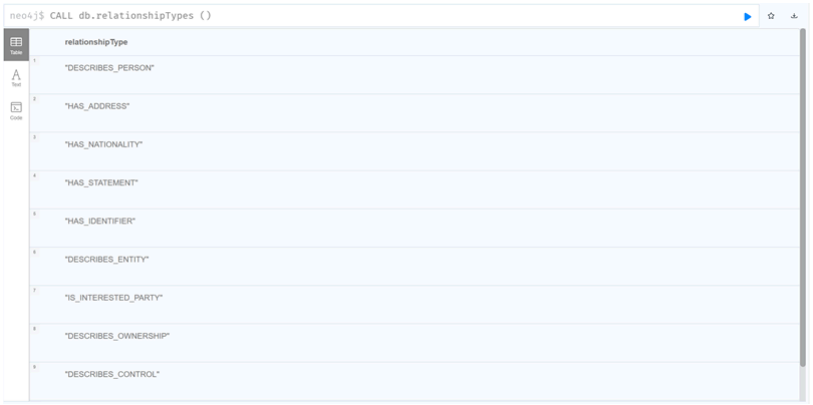
Figur 40: PersonStatement peker mot Person

Resultatet viste at Person-noden, relasjonen og PersonStatement-noden alle var til stede og korrekt koblet. PersonStatement-noden inneholder riktig metadata som statementID, pub_date, statementType og publiseringsinformasjon fra OpenOwnership Register.

Relasjonstypene i databasen ble verifisert ved å kjøre:

```
CALL db.relationshipTypes ()
```

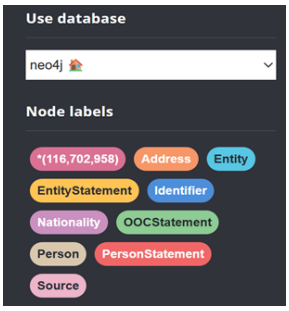
Dette bekrefter at alle 10 forventede relasjonstyper var til stede: DESCRIBES_PERSON, HAS_ADDRESS, HAS_NATIONALITY, HAS_STATEMENT, HAS_IDENTIFIER, DESCRIBES_ENTITY, IS_INTERESTED_PARTY, DESCRIBES_OWNERSHIP, DESCRIBES_CONTROL og INVESTIGATION_LINK.



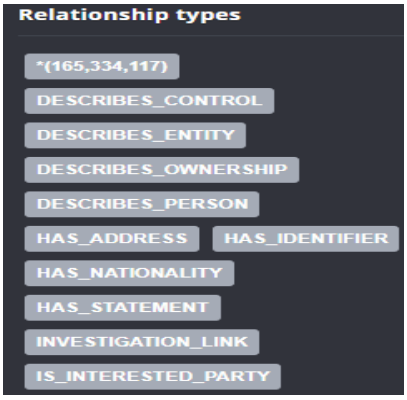
Figur 41: Databaseverifisering

7.3 Importkontroll

Etter fullført import ble det verifisert at databasen inneholdt totalt 116 702 958 noder og 165 334 117 relasjoner, fordelt på ni nodetyper og ni relasjonstyper i samsvar med den definerte datamodellen.



Figur 42: Node Labels



Figur 43: Relationship types

Indeksene ble kontrollert ved å kjøre SHOW INDEXES, som bekreftet at alle indekser hadde status ONLINE og 100% populeringsgrad. Dette inkluderer indekser på blant annet person_personID, person_statement_id og ooc_statement_ident.



neo4j$ SHOW INDEXES

	id	name	state	populationPercent	type	entityType	labelsOrTypes	properties	indexProvider	owningConstraint	lastR
1	13	"address_address"	"ONLINE"	100.0	"RANGE"	"NODE"	["Address"]	["address"]	"range-1.0"	"address_address"	
2	5	"entity_entityID"	"ONLINE"	100.0	"RANGE"	"NODE"	["Entity"]	["entityID"]	"range-1.0"	"entity_entityID"	
3	11	"entity_statement_id"	"ONLINE"	100.0	"RANGE"	"NODE"	["EntityStatement"]	["statementID"]	"range-1.0"	"entity_statement_id"	
4	15	"identifier_id"	"ONLINE"	100.0	"RANGE"	"NODE"	["Identifier"]	["id"]	"range-1.0"	"identifier_id"	
5	1	"index_343aff4e"	"ONLINE"	100.0	"LOOKUP"	"NODE"	null	null	"token-lookup-1.0"	null	
6	2	"index_f7700477"	"ONLINE"	100.0	"LOOKUP"	"RELATIONSHIP"	null	null	"token-lookup-1.0"	null	
7	19	"inv_link_entity"	"ONLINE"	100.0	"RANGE"	"RELATIONSHIP"	["INVESTIGATION_LINK"]	["sharedEntityID"]	"range-1.0"	null	
8	7	"ooc_statement_ident"	"ONLINE"	100.0	"RANGE"	"NODE"	["OOCStatement"]	["statementID"]	"range-1.0"	"ooc_statement_ident"	

Figur 44: Indeks-kontroll

Det ble også bekreftet at sirkularitetsdeteksjon ikke avdekket noen eierlooper i det importerte datasettet, det vil si ingen selskaper eier seg selv gjennom indirekte kjeder.



Figur 45: Sirkularitetsdeteksjon

7.4 Ytelsestest

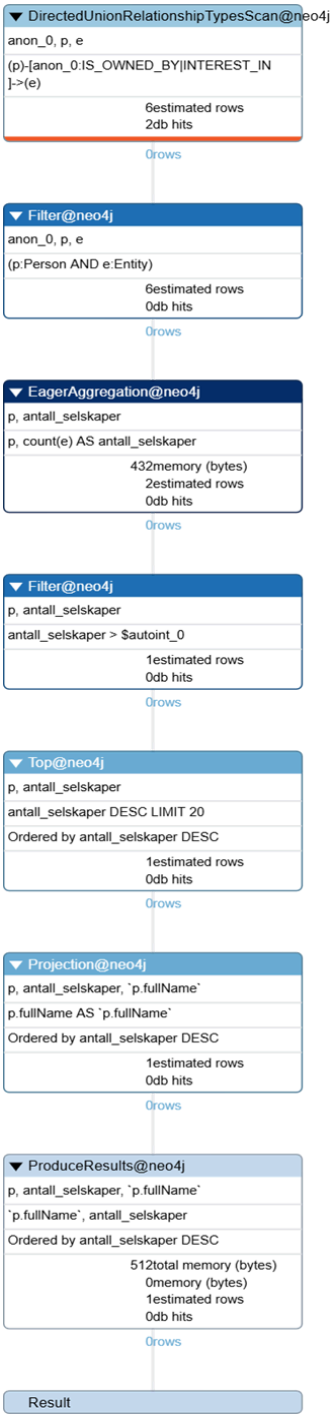
Ytelsestestingen undersøkte responstid på analytiske Cypher-spørringer ved bruk av PROFILE-nøkkelordet, som gir detaljert informasjon om antall databasetilganger (db hits) og minnebruk.

Første test uten indeks på personID viste at spørringer over alle Person-noder tok svært lang tid. En spørring for å finne personer med flest selskapsskoblinger brukte over 736 000 millisekunder og returnerte ingen resultater.

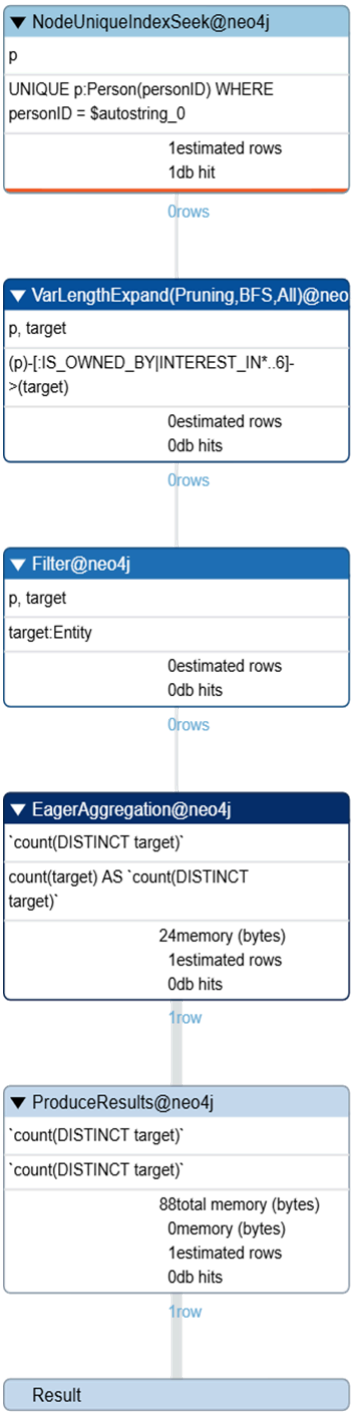
Etter at indeks ble opprettet:

CREATE INDEX person_id_idx IF NOT EXISTS FOR (p: Person) ON (p.personID)

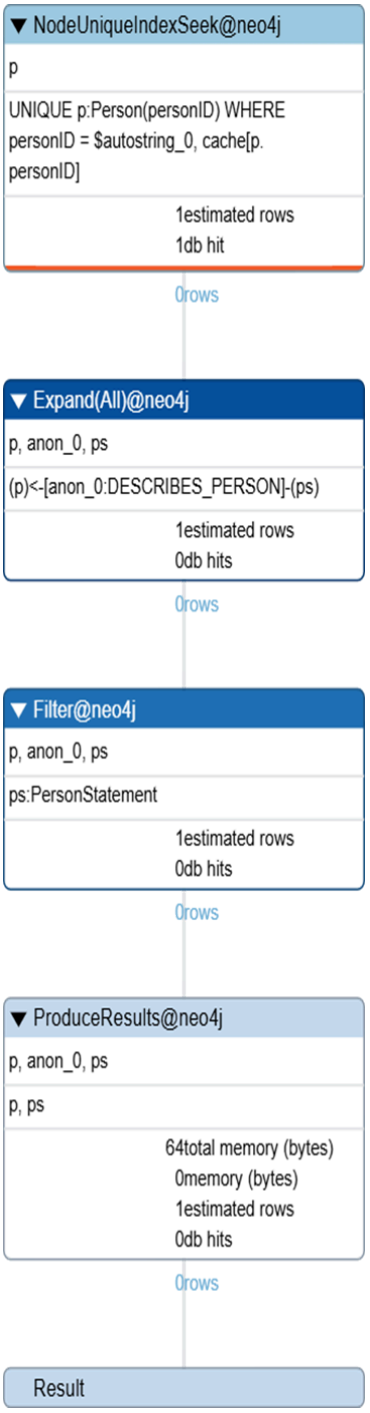
Ble spørringer mot spesifikke Person-noder via personID drastisk raskere. PROFILE- planen viste at planen nå startet med en NodeUniqueIndexSeek-operasjonen med kun 1-2 dB hits, i stedet for en full NodeByLabelScan over alle noder.



Figur 46: Plan 1



Figur 47: Plan 2



Figur 48: Plan 3

En variabel-dybde traversering med opptil seks relasjoner for en spesifikk person fullførte på 53-55 millisekunder etter at indeksen var på plass, noe som er akseptabelt for interaktiv bruk.

7.5 Aggregeringstest

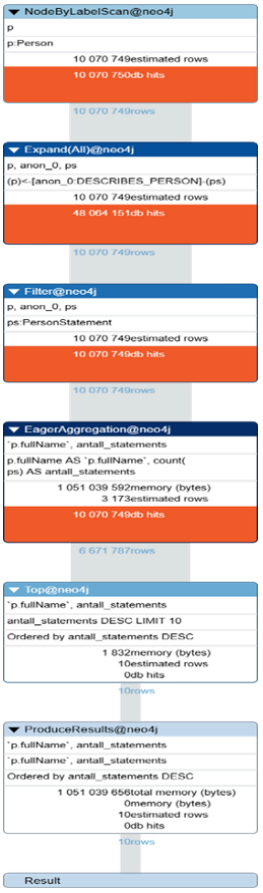
Aggregeringstestene verifiserte at sammenstilling av data på tvers av mange noder ga korrekte resultater. Spørringen under teller antall PersonStatement-noder per Person og returnerer de ti med flest, med relasjonen i korrekt retning fra PersonStatement mot Person:

```
MATCH (ps:PersonStatement)-[:DESCRIBES_PERSON]->(p:Person)
RETURN p.fullName, count(ps) AS antall_statements
ORDER BY antall_statements DESC
LIMIT 10
```

PROFILE- analysen viste en total utførelsestid på 29109 millisekunder med 78 276 399 db hits. Planen traverserte alle 10 070 749 Person-noder via NodeByLabelScan, etterfulgt av Expand(All) med 48 064 151 db hits.



Figur 49: Aggregeringstest med tid



Figur 50: Aggregeringstest med tabell

```
MATCH (ps:PersonStatement)-[:DESCRIBES_PERSON]→(p:Person)
RETURN p.fullName, count(ps) AS antall_statements
ORDER BY antall_statements DESC
LIMIT 10
```

	p.fullName	antall_statements
1	"Darren Symes"	596
2	"Imran Khan"	591
3	"Gurpreet Singh"	589
4	"Jaroslav Haščák"	523
5	"Marek Dospiva"	516
6	"Muhammad Ali"	514
7	"David Smith"	508
8	"Muhammad Usman"	491

Figur 51: Resultat topp 10 funn

Spørringen returnerte korrekte resultater med Darren Symes på topp med 596 registreringer, fulgt av Imran Khan (591) og Gurpreet Singh (589).

En tilsvarende aggregeringsspørring for eierskapsforhold:

```
MATCH (p:Person)-[:IS_INTERESTED_PARTY]→(ooc:OOCStatement)-[:DESCRIBES_OWNERSHIP]→(e:Entity)
WITH p, count(e) AS antall_selskaper
WHERE antall_selskaper > 1
RETURN p.fullName, antall_selskaper
ORDER BY antall_selskaper DESC
LIMIT 20
```

	p.fullName	antall_selskaper
1	"Darren Symes"	19734
2	"Peter Valaitis"	9821
3	"Darren Symes"	8299
4	"Michael Gleissner"	7558
6	"Terry Dunne"	5763
6	"Peter Valaitis"	5160
7	"Mohammed Ayyaz"	4276
8	"Grzegorz Szewczyk"	4194

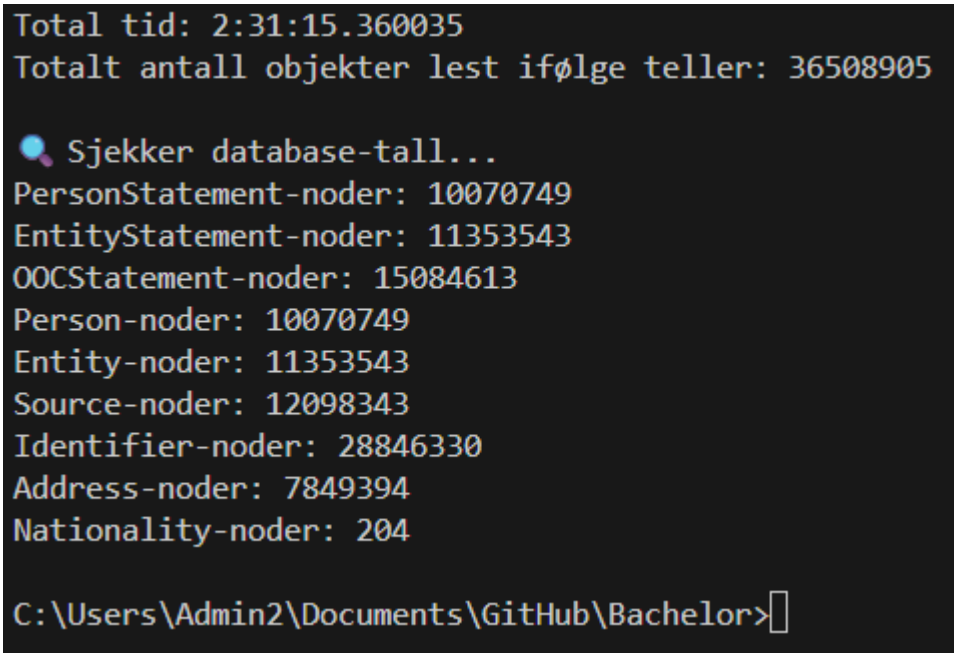
Started streaming 20 records after 10 ms and completed after 39892 ms.

Figur 52: Cypher-spørring for eierskapsforhold

Denne fullførte på 10 millisekunder og returnerte eierskapsforhold og fulgte den faktiske datamodellen der Person er koblet til Entity via OOCStatement gjennom IS_INTERESTED_PARTY og DESCRIBES_OWNERSHIP.

7.6 Ytelsestest av importprosessen

Ytelsen av importen av nodene og relasjonene inn i Neo4j ble dokumentert for å finne det mest effektive oppsettet. For nodeimporten ble importen kjørt to ganger, en gang med 8 arbeidere og en gang med 12 arbeidere for å sjekke forskjellen i tidsbruk. Med 8 arbeidere tok importen av nodene 2 timer og 31 minutter som kan ses under figur (53)



Figur 53: Import av noder med 8 arbeidere

Det ble totalt importert 106 millioner noder inn i Neo4j. Databasen ble slettet og koden ble kjørt på nytt med noen små endringer og 12 arbeidere, da tok det 43 minutter , se figur (54)

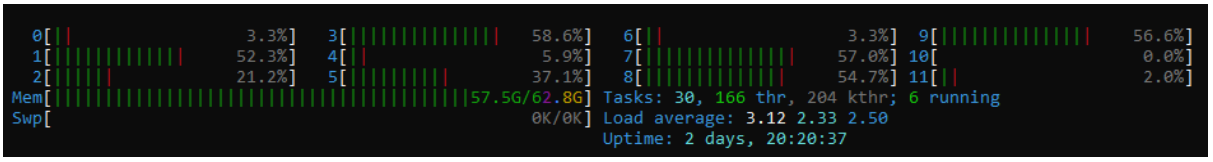
```
🎉 Fase 1 ferdig (Alle noder opprettet)!
Total tid: 0:43:53.023913
Totalt antall objekter lest ifølge teller: 36508905

🔍 Sjekker database-tall...
PersonStatement-noder: 10070749
EntityStatement-noder: 11353543
OOCStatement-noder: 15084613
Person-noder: 10070749
Entity-noder: 11353543
Source-noder: 12098343
Identifier-noder: 38933639
Address-noder: 7737575
Nationality-noder: 204
```

Figur 54: Import av Noder med 12 arbeidere

Det ble importert ca 116 millioner noder når koden ble kjørt med 12 arbeidere. Dette er på grunn av en kodefeil som gjorde at 10 millioner identifier-noder manglet når importen ble kjørt med 8 arbeidere. Men til og med når det var 10 millioner flere noder var importen mye mer effektiv med 12 arbeidere istedenfor 8. Grunnen til dette er siden at nodeimport til Neo4j er primært CPU-bound. Dette betyr at jo flere kjerner/arbeidere man har, jo raskere vil det gå i teorien hvis man også har nok minne.

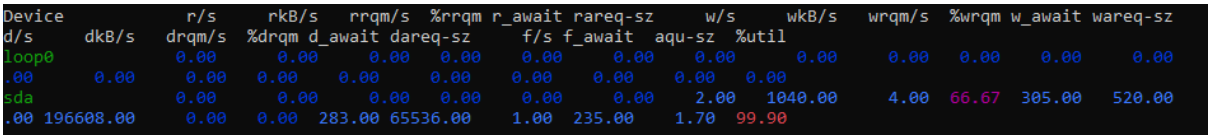
Ressursbruken ble hele tiden overvåket i Linux-terminalen via kommandoen iostat og kan ses under figur (55)



Figur 55: Ressursbruk Nodeimport med 12 arbeidere

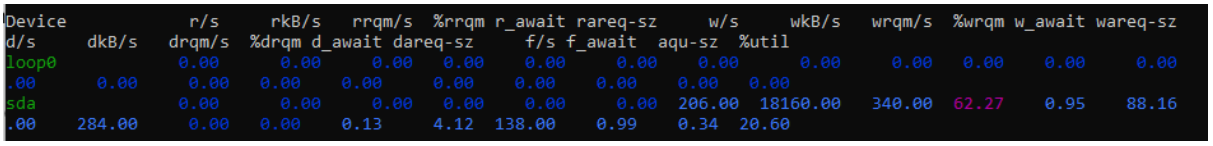
Minnebruket økte gradvis opp til og holdt seg stabil på 57,5 GB, dette er siden Neo4j hadde fått tildelt 16 GB heap og 40 GB page cache. Dette gir totalt 56 GB, de resterende 1,5 GB kommer fra OS.

Relasjonsimporten ble bare kjørt en gang og bare med 6 arbeidere, grunnen til dette var for å redusere deadlocks og holde importen stabil. Relasjonsimporten er primært I/O bound altså basert på harddiskhastigheten, siden databasen må finne nodene i databasen og koble de korrekt. Kommandoen iostat -dx 1 ble kjørt for å sjekke diskbruken figur (56)



Figur 56: Høyt diskbruk

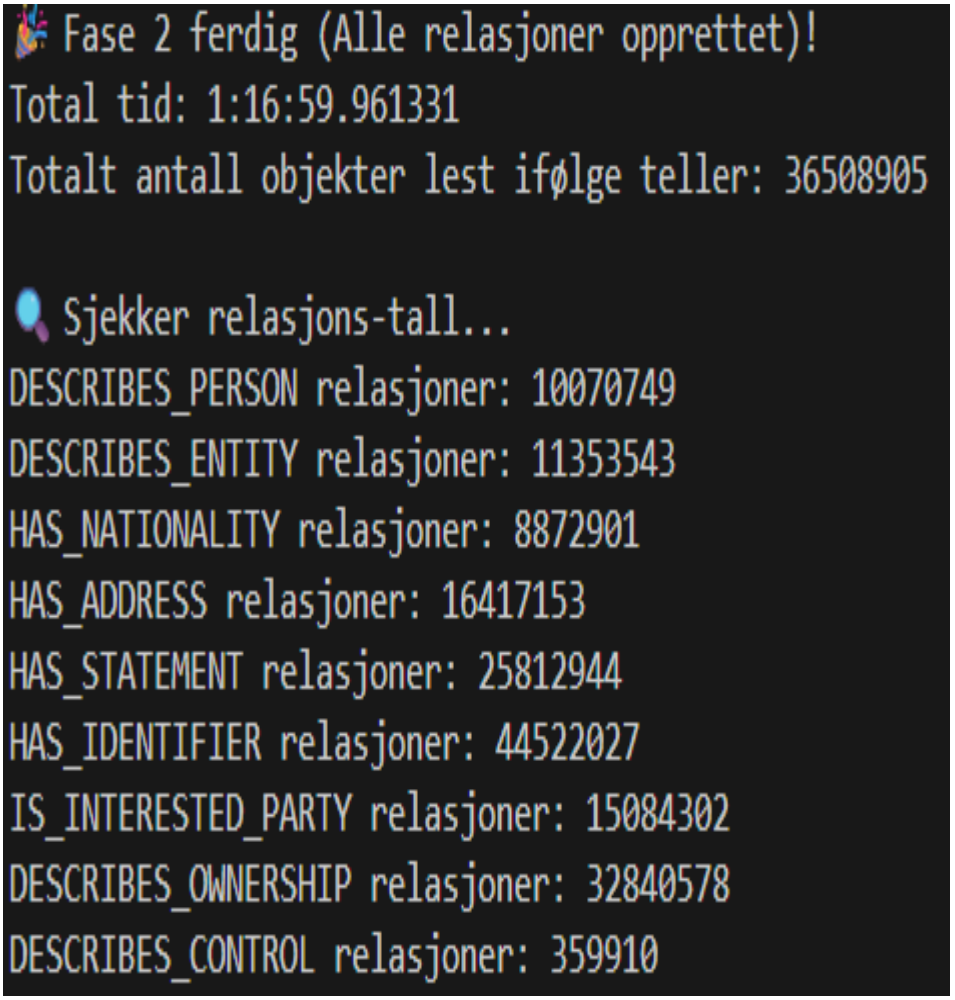
Her ser man at diskutnyttelsen var på 99.90% som er ganske makset ut som gjør harddisken til en bottleneck. Heldigvis gikk dette ned og stabiliserte seg ganske raskt figur (57)



Figur 57: Normalt diskbruk

Grunnen til at diskbruket var så høyt i starten var nok siden det tar litt tid før det stabiliserer seg og når steady-state.

Fase 2 relasjonsimporten for ca 165-millioner relasjoner tok ca 1 time og 17 minutter figur (58)



Figur 58: Relasjonsimporten

Totalt tok importen av noder og relasjoner ca 2 timer på gruppens VM. Importen kan gå raskere/tregere avhengig av hvor mye datakraft man har tilgjengelig.

8 Brukerdokumentasjon

8.1 Systemoversikt

Systemet gjør det mulig å importere datasettet fra OpenOwnership til grafdatabase i Neo4j, og samtidig analysere eierskapsstrukturer gjennom et REST API utviklet med FastApi. Løsning består av følgende deler:

- Parsing av JSON-data fra OpenOwnership.
- Import av noder og relasjoner til Neo4j.
- API for søk og analyse av eierskap.
- Logging og autentisering for brukere.

8.2 Installasjon og oppsett

For å kjøre systemet, må nødvendige avhengigheter være installert, den virtuelle maskinen (VM) startes, Neo4j konfigurert og API-et kjøres.

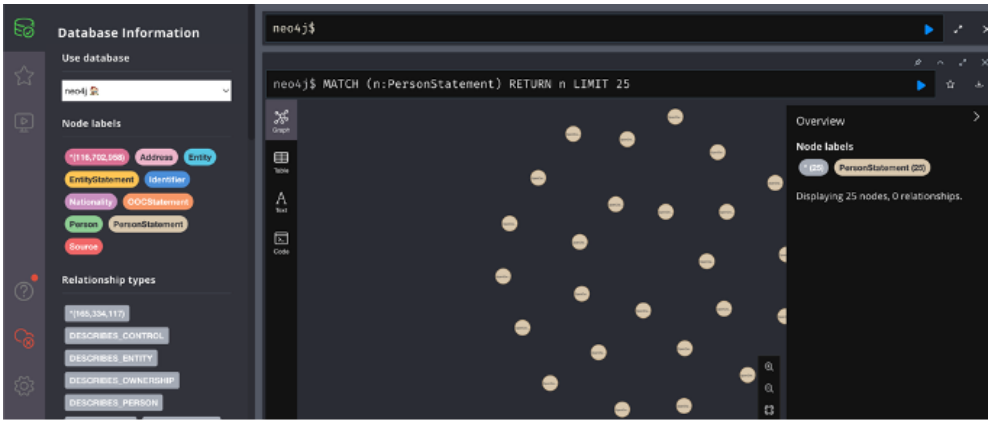
Avhengigheter kan installeres med:

```
pip install -r requirements.txt
```

Se kapittel 2.3.2 for oversikt over avhengigheter.

8.2.1 Oppstart av Neo4j

Neo4j må være startet for at API-et kan koble seg til databasen. Databasen og API-et kommuniserer med Bolt-protokollen. Ved oppstart verifiserer API-et tilkoblingen til Neo4j med funksjonen *verify_connectivity()*. Neo4j fungerer som en systemtjeneste på VM.



Figur 59: Neo4j browser etter oppstart.

8.2.2 Oppstart av API

Etter at Neo4j er startet kan API-et kjøres fra prosjektmappen i Python sitt virtuelle miljø. API-et startes med hjelp av uvicorn:

```
Uvicorn main:app --host 0.0.0.0 --port 8000 -reload
```

Når API-et starter, kobler systemet til Neo4j, indekser opprettes og API endepunktene blir tilgjengelige.

8.3 Bruk av systemet

8.3.1 Import av data

Importen av dataen fra OpenOwnershop Register gjennomføres i to omganger, med import av noder og deretter relasjonene mellom nodene. Dette ble gjort for å sikre at noder eksisterte i Neo4j før relasjoner ble lagt til mellom nodene. Datasettet ble lest fra JSON-filen linje for linje. Hver linje er et eget JSON-objekt, som ble parset med *orjson*, og deretter klassifisert med feltet *statementType*. Datasettet blir så transformert til noder i Neo4j. Etter dette blir relasjoner opprettet mellom de eksisterende nodene. For å håndtere det store datasettet blir disse importene gjort med batching og parallellisering.

Etter importen kjøres kontrollspørringer for å få oversikt over antall noder og relasjoner. Dette gjøres for å verifisere at importen er gjennomført som forventet.

8.3.2 Bruk av API

Systemet tilbyr et REST API utviklet med FastApi. Api-et brukes for å hente ut data fra Neo4j-databasen, uten at man trenger å åpne Neo4j browseren direkte. API-et tilbyr: søk av personer, søk av selskaper, analyse av eierskap og kontrollforhold og opprettelse av etterforskningskoblinger.

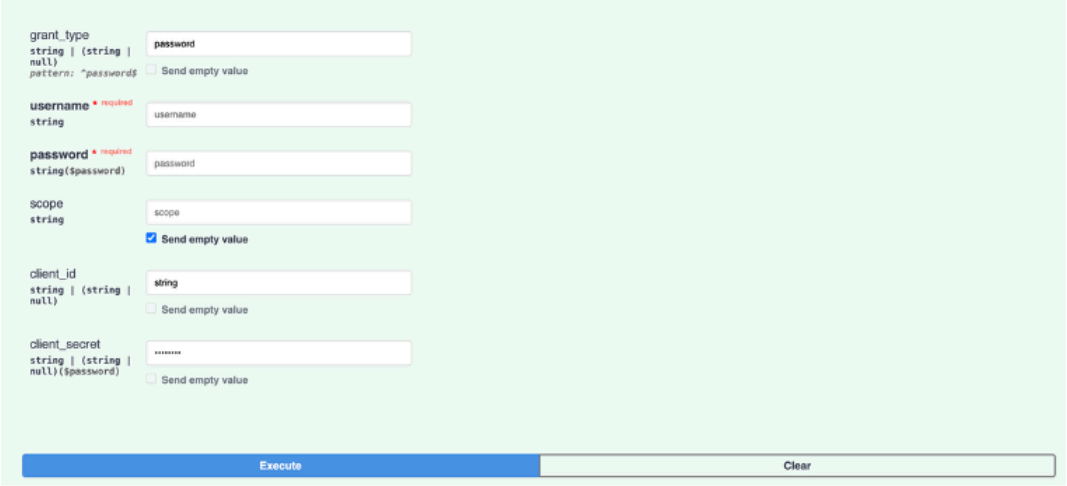
API-et er tilgjengelig gjennom Swagger UI, som genereres med FastApi automatisk. Her får brukeren oversikt over alle endepunktene. Se figur 33.

8.3.3 Innlogging og autentisering

Flere av endepunktene er beskyttet og krever autentisering. For å bruke disse må man logge inn med endepunktet POST /token, og for å kunne logge inn må man registrere seg med endepunktet POST /register.

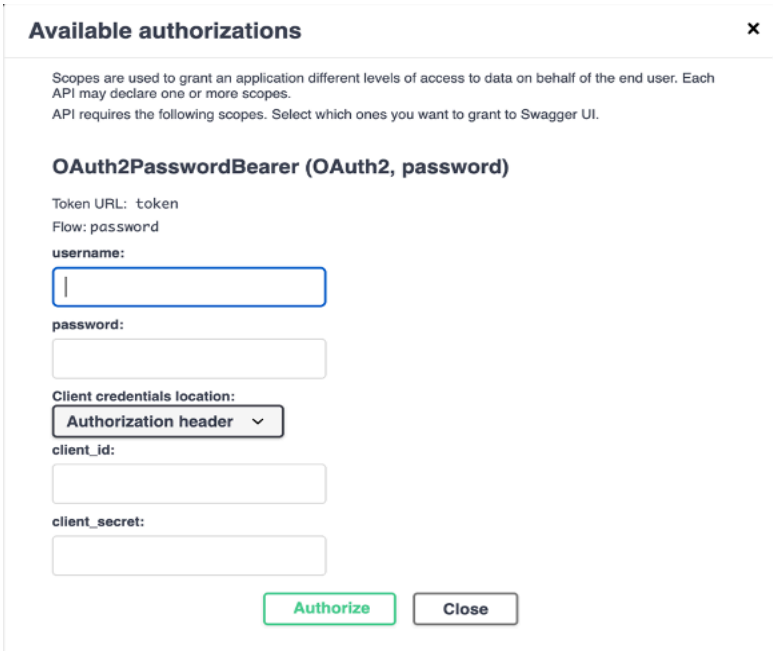
Ved innlogging sender brukeren inn brukernavn og passord, og ved vellykket innlogging returnerer API-et en JWT-token. Token brukes for autorisering ved senere forespørsler.
I Swagger UI gjøres dette med knappen Authorize, hvor tokenet legges til før brukeren tester endepunktet.

Autentisering med POST /token:



Figur 60: POST /token, innlogging i Browser

Autentisering med Autherize med Swagger UI.



Figur 61: Innlogging med autogenerert Authorization.

8.3.4 Eksempel på respons

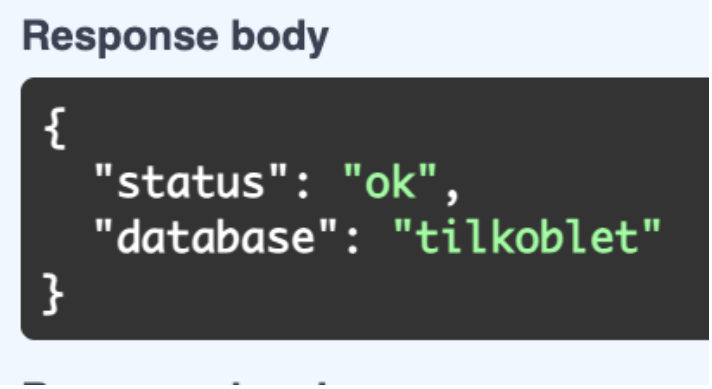
Ved bruk av endepunktet GET /Person/{personID}/detaljer vil brukeren kunne skrive inn en *personID* og få ut følgende respons:



Figur 62: Formattering på svar på API-et

8.3.5 Status-endepunkt

Systemet har et eget endepunkt for health-check. Dette brukes for å kontrollere at API-et og databasen fungerer som forventet. Endepunktet er GET /helse, og ved å kjøre endepunktet får man følgende respons:

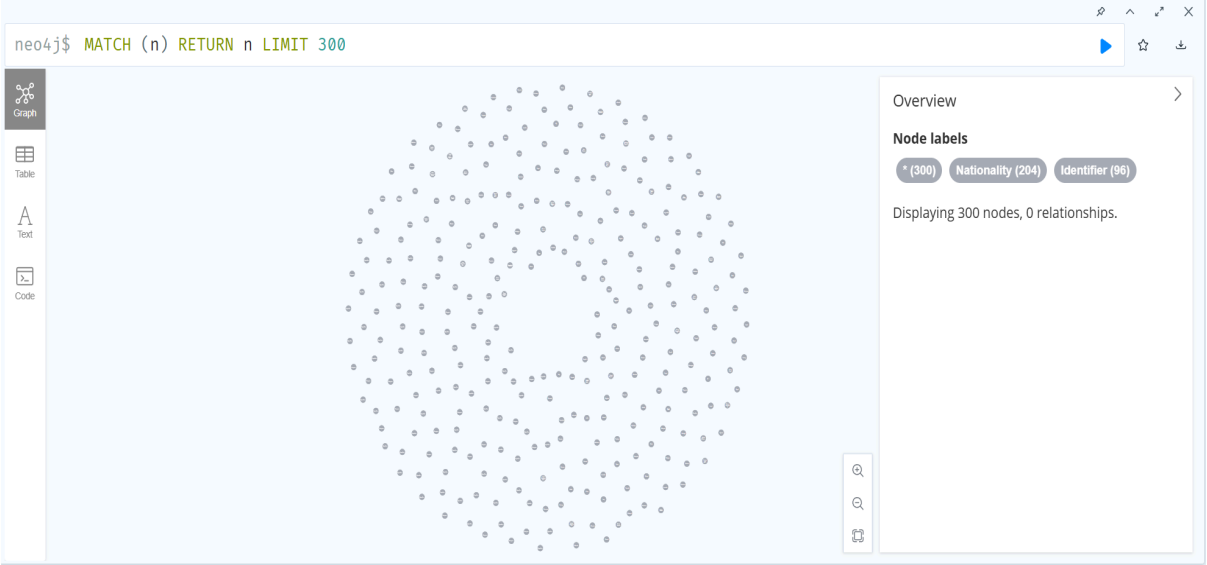


Figur 63: Respons fra Status endepunktet.

8.4 Bruk av Neo4j Browser

Neo4j browser er et verktøy som kan brukes til å visualisere og analysere data som er i grafdatabasen. Dette gjør det mulig å få en større forståelse for hvordan nodene og relasjonene er strukturert i datasettet (Neo4j, u.å.-e).

Man kan kjøre Cypher-spørringer direkte i Neo4j-browser for å hente ut og analysere data. Et eksempel på en spørring er MATCH (n) RETURN n LIMIT 300 Denne spørringen gir et lite utvalg av nodene i databasen figur ()



Figur 64 : Et lite utvalg av noder

Denne spørringen viste bare Nationality og Identifier-noder siden utvalget er veldig lite i forhold til alle de 116 millioner-nodene i datasettet.

I prosjektet er Neo4j-browser brukt til å teste og validere spørringer som er spesielt utviklet for å analysere eierskapsstrukturer. Dette fører til at det blir enklere med å feilsøke og sikre at spørringene gir de forventende resultatene før de implementeres i API-et.

8.5 Feilhåndtering

Systemet ble utviklet med grunnleggende feilhåndtering for å sikre stabil drift og tilbakemelding dersom feil oppstår. Dette gjør at flere vanlige feilsituasjoner blir fanget opp og håndtert på en kontrollert måte.

Dersom en datafil mangler eller har et ugyldig format vil systemet merke det og importprosessen vil stoppe og en feilmelding blir generert. Hvis databasen er utilgjengelig vil API-et returnere en feilrespons til klienten. API-et benytter også HTTP-statuskoder for å indikere ulike typer feil, for eksempel ved ugyldige forespørsler eller interne serverfeil.

Alle feil i API-et blir loggført via systemets loggfunksjon. Dette gjør det mulig å analysere i etterkant og forbedre stabiliteten til systemet.

8.6 Begrensninger

Systemet har noen bevisste begrensninger som følge av designvalg og implementasjon. Det finnes ikke et eget frontend siden politiet skal implementere API-endepunktene i sitt eget frontend. Alt av interaksjon med databasen skjer via Neo4j-Browser eller API-et.

API-et inneholder bare et sett med forhåndsdefinerte spørringer, dette reduserer fleksibiliteten for brukere som ønsker mer fleksibilitet. Systemet er også avhengig av et lokalt kjørende Neo4j-instans som fører til at ytelse og tilgjengelighet er avhengig av denne installasjonen.

Ytelsen vil variere i forhold til hvor mye ressurser som er tilgjengelig på den virtuelle maskinen som Neo4j-databasen kjører på. Ytelsen vil ha mer å si når det er komplekse og store spørringer som henter ut mye data og krever mye ressurser.

9 Avslutning

For å vurdere løsningen på en strukturert måte er det nyttig å sammenligne resultatene med kravspesifikasjonen som ble definert i begynnelsen av prosjektet.

Funksjonelle krav

De fleste funksjonelle kravene ble oppfylt. F1 ble innfridd ved at systemet leste og behandlet data fra OpenOwnership Register fra en fil. F2 ble oppfylt ved å representere individer og selskaper som noder i Neo4j, med eierskapsforhold representert som relasjoner. F3 blir oppfylt gjennom Neo4j-nettleseren, som gir visuell fremstilling av noder og relasjoner. F4 er innfridd ved bruk av Cypher-spørringer som avslører direkte og indirekte eierskaps- og kontrollkjeder.

F5, kravet om at systemet skal kunne kjøres flere ganger uten å opprette dupliserte noder eller relasjoner, er delvis oppfylt. Noder håndteres ved hjelp av MERGE, men relasjonstypen DESCRIBES_OWNERSHIP benytter CREATE i stedet for MERGE. Dette valget ble tatt fordi bruk av MERGE resulterte i tap av over én million relasjoner under importen. Konsekvensen er at gjentatte kjøringer kan produsere dupliserte relasjoner. Dette representerer en kjent svakhet i løsningen.

F6 og F7 er oppfylt gjennom FastAPI, som tilbyr forhåndsdefinerte endepunkter for søk etter personer, selskaper og eierskapsforhold uten at brukeren trenger direkte tilgang til Neo4j Browser. F8 oppfylles med et eget helse-endepunkt (GET /helse) som bekrefter at tjenesten og databaseforbindelsen fungerer. Det valgfrie kravet F9 om støtte for visualiseringsverktøy som NeoDash ble ikke prioritert og er derfor ikke gjennomført.

Ikke-funksjonelle krav

NF1 og NF2 om stabilitet og robusthet er oppfylt. Systemet håndterte et datasett på omtrent 38 GB og over 36 millioner JSON-linjer uten kritiske feil, og ufullstendige objekter ble ignorert uten at prosessen stoppet.

NF3 angående reproduserbarhet er delvis oppfylt på grunn av problemer relatert til DESCRIBES_OWNERSHIP. NF4 og NF5 er innfridd ved at koden er dokumentert, og at API-en automatisk genererer dokumentasjon gjennom Swagger UI og OpenAPI-standarden.

Sikkerhetskrav

Sikkerhetskravene S1 til S5 ble overholdt. Systemet bruker kun åpne og offentlig tilgjengelige data, og ingen sensitive personopplysninger lagres utover det som finnes i datasettet. API-en er kun tilgjengelig internt på den virtuelle maskinen, og autentisering er implementert med JWT og bcrypt-hashing av passord.

Ytelseskrav

Y1 til Y4 ble innfridd. Importprosessen ble fullført innen rimelig tid på tilgjengelig maskinvare, og interaktive spørringer via indekser besvares på få millisekunder. Den virtuelle maskinen med 64 GB RAM viste seg tilstrekkelig for å håndtere datasettet i Neo4j.

Kvalitetskrav

K1 til K4 er oppfylt ved at grafmodellen er intuitiv, spørringer er dokumentert, og koden er strukturert i logiske moduler for å sikre vedlikeholdbarhet.

Tekniske rammer

Alle tekniske krav T1 til T7 ble fulgt. Neo4j Community Edition ble benyttet som database, Python som hovedspråk, og Cypher for modellering og spørringer. Systemet kjører på en Linux-basert virtuell maskin via Pegasus. Indekser og constraints er implementert, og API-et kommuniserer med Neo4j via Bolt-protokollen gjennom den offisielle Python-driveren.

Et punkt til refleksjon er den hybride bruken av Rust for importprosessen. Dette ga betydelige ytelsesforbedringer, men økte samtidig kompleksiteten i kodebasen. En ren Python-løsning ville vært enklere å vedlikeholde, men med betydelig lengre importtid.

9.1 Diskusjon

Prosjektet viser at store og komplekse eierskapsstrukturer kan modelleres effektivt ved bruk av grafdatabaser. Neo4j viste seg å være godt egnet til formålet, spesielt fordi grafmodellen gjør det mulig å traversere relasjoner mellom personer og selskaper på en måte som ville vært langt mer komplisert i en tradisjonell relasjonsdatabase. Dette gjør løsningen relevant for analyse- og etterforskningsarbeid hvor det er viktig å identifisere indirekte forbindelser og kontrollkjeder.

Databasen inneholder over 116 millioner noder og 165 millioner relasjoner fordelt på ni nodetyper og ti relasjonstyper. Ytelsestesting viser at systemet håndterer store mengder data tilfredsstillende, og spørringer mot spesifikke aktører via indekser besvares interaktivt på noen få millisekunder.

Bruken av parallellprosessering og en Rust-basert Neo4j-driver reduserte importtiden betydelig sammenlignet med en ren Python-basert løsning. Samtidig introduserte denne hybride tilnærmingen økt kompleksitet i kodebasen. Selv om ytelsesgevinsten var betydelig, kan kombinasjonen av Python og Rust gjøre systemet mer krevende å vedlikeholde og videreutvikle over tid.

Et sentralt kompromiss i løsningen er håndteringen av relasjonstypen DESCRIBES_OWNERSHIP. For å unngå tap av et stort antall relasjoner ble CREATE benyttet i stedet for MERGE. Dette løste utfordringen med manglende relasjoner under import, men samtidig betyr det at gjentatte kjøringer kan føre til duplisering av relasjoner. Derfor oppfyller ikke løsningen fullt ut kravet om fullstendig reproduserbarhet, og dette representerer en kjent svakhet som bør tas opp i videre arbeid.

Det er også viktig å vurdere begrensningene i databasen. OpenOwnership-Register i denne løsningen inneholder kun data fra Danmark, Slovakia og Storbritannia. Dette begrenser systemets nytteverdi i etterforskning som involverer andre jurisdiksjoner. Videre varierer kvaliteten og konsistensen i datasettene, noe som påvirker hvor pålitelige analysene kan være. Systemet er derfor avhengig av at datakilden blir vedlikeholdt og oppdatert over tid.

API-et dekker de sentrale behovene som er definert i kravspesifikasjonen, men funksjonaliteten er fortsatt relativt begrenset. Endepunktene er forhåndsdefinerte og gir ikke brukeren full fleksibilitet til å utføre egne analyser. Mangelen på et frontend gjør også at terskelen for bruk er høyere enn ønskelig, spesielt for brukere uten teknisk kompetanse innen API-er eller Cypher-spørringer.

Til tross for disse begrensningene viser prosjektet at grafdatabaser er et effektivt verktøy for modellering og analyse av komplekse eierskapsstrukturer. Løsningen gir et godt grunnlag for videre utvikling, både når det gjelder funksjonalitet, datagrunnlag og brukergrensesnitt.

9.2 Konklusjon

Prosjektet har besvart problemstillingen ved å modellere data fra OpenOwnership Register som en grafdatabase i Neo4j, og gjort det tilgjengelig gjennom et applikasjonslag basert på FastAPI. Løsningen gjør det mulig å analysere direkte og indirekte eierskaps- og kontrollstrukturer mellom enkeltpersoner og selskaper på en effektiv måte.

Systemet håndterer store datamengder og oppnår tilfredsstillende ytelse både under import og ved analysespørringer. Bruken av grafdatabase og Cypher-spørringer viste seg å være spesielt godt egnet for å analysere komplekse relasjoner i datasettet.

Kravspesifikasjonen ble i stor grad oppfylt. Hovedunntakene er den valgfrie NeoDash-visualiseringen og utfordringen knyttet til duplisering av relasjoner ved gjentatte import. Til tross for dette vurderes løsningen som funksjonell, stabil og egnet som grunnlag for videre utvikling.

Prosjektet har gitt gruppen verdifull erfaring med datamodellering, ytelsesoptimalisering og API-utvikling i et reelt oppdrag med praktisk nytteverdi.

9.3 Videre arbeid

Det finnes flere naturlige retninger for videre utvikling av systemet. Det mest åpenbare steget er utvikling av et frontend som gjør systemet enklere å bruke for

etterforskere uten teknisk kompetanse innen API-er eller Cypher-spørringer. En grafbasert visualisering av eierskapsnettverk kan betydelig forbedre brukervennligheten.

API-et bør også utvides med flere og mer fleksible endepunkter. Den nåværende løsningen tilbyr hovedsakelig forhåndsdefinerte søk, mens fremtidige brukere kan ha behov for mer dynamiske søkealternativer, for eksempel filtrering etter nasjonalitet, tidsperiode eller eierandel.

Datagrunnlaget kan videre styrkes ved å inkludere flere land og datakilder utover de tre jurisdiksjonene som brukes i dag. En slik utvidelse vil øke systemets relevans for analyse av grenseoverskridende eierskapsstrukturer.

Til slutt bør det etableres rutiner for automatisk oppdatering av datasettet. I den nåværende løsningen utføres importen manuelt. Et system for periodisk innsamling og trinnvis oppdatering av grafen vil bidra til å sikre at datagrunnlaget forblir oppdatert over tid.

Kildeliste

Bauer, R. (2024, September 4). *Python driver 10x faster with Rust*. Neo4j. https://neo4j.com/blog/developer/python-driver-10x-faster-with-rust/

Department of Computer Science, Oslo Metropolitan University. (2024). *Pegasus Cloud Service*. https://docs.sky.oslomet.no/

FastAPI. (u.å.-a). *FastAPI framework, high performance, easy to learn, fast to code, ready for production*. Hentet 26. april 2026 fra https://fastapi.tiangolo.com/

FastAPI. (u.å.-b). *First steps*. Hentet 5. mars 2026 fra https://fastapi.tiangolo.com/tutorial/first-steps/#operation

FastAPI. (u.å.-c). *Interactive API docs*. Hentet 5. mars 2026 fra https://fastapi.tiangolo.com/#interactive-api-docs

Git. (u.å.). *1.3 Getting Started - What is Git?* Hentet 17. april 2026 fra https://git-scm.com/book/en/v2/Getting-Started-What-is-Git%3F

GitHub. (u.å.-a). *About GitHub and Git*. Hentet 17. april 2026 fra https://docs.github.com/en/get-started/start-your-journey/about-github-and-git

GitHub. (u.å.-b). *About pull requests*. Hentet 17. april 2026 fra https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-pull-requests

Internet Archive. (2026). *Archived snapshot of OpenOwnership BODS register*. https://web.archive.org/web/20251127040344/https://bods-data.openownership.org/source/register/

Microsoft. (2025, October 22). *Introduction to APIs*. Microsoft Learn. https://learn.microsoft.com/en-us/xandr/industry-reference/intro-to-apis

Microsoft. (u.å.-a). *Visual Studio Code documentation*. Hentet 16. april 2026 fra https://code.visualstudio.com/docs

Microsoft. (u.å.-b). *Hva er en virtuell maskin?* Hentet 22. april 2026 fra https://azure.microsoft.com/nb-no/resources/cloud-computing-dictionary/what-is-a-virtual-machine/

Mishra, S. (2023, 22. november). *Rust vs Python: Language of the Future?* https://no.linkedin.com/pulse/rust-vs-python-language-future-siddharth-mishra-om2if

Neo4j. (2025, July 15). *What is a graph database?* https://neo4j.com/blog/graph-database/what-is-a-graph-database/

Neo4j. (u.å.-a). *What is neo4j?* Hentet 4. mars 2026 fra https://neo4j.com/docs/getting-started/whats-neo4j/

Neo4j. (u.å.-b). *Neo4j Graph Database Platform*. Hentet 15. mars 2026 fra https://neo4j.com/product/neo4j-graph-database/

Neo4j. (u.å.-c). *Cypher Manual*. Hentet 16. mars 2026 fra https://neo4j.com/docs/cypher-manual/current/

Neo4j. (u.å.-d). *Introduction to Neo4j: What is it and why is it important for software engineering?* Hentet 26. april 2026 fra https://neo4j.app/article/Introduction_to_Neo4j_What_is_it_and_why_is_it_important_for_software_engineering.html

Neo4j. (u.å.-e). *Neo4j Browser documentation.* Hentet 5. mai 2026 fra https://neo4j.com/docs/browser/

OpenOwnership. (2026). *Beneficial Ownership Data Standard (BODS) register*. Hentet 20. mars 2026 fra https://bods-data.openownership.org/source/register/

OpenOwnership. (u.å.). *What we do*. Hentet 12. mars 2026 fra https://www.openownership.org/en/about/what-we-do/

Politiet. (u.å.). *Politiets IT-enhet. Om oss*. Hentet 18. april 2026 fra https://www.politiet.no/om-politiet/organisasjonen/sarorganene/pitprg

PRG. (u.å.). *Hva er Rust?* Hentet 23. april 2026 fra https://prg.no/lang/rust/

Python. (u.å.). *What is Python? Executive Summary*. Hentet 15. april 2026 fra https://www.python.org/doc/essays/blurb/

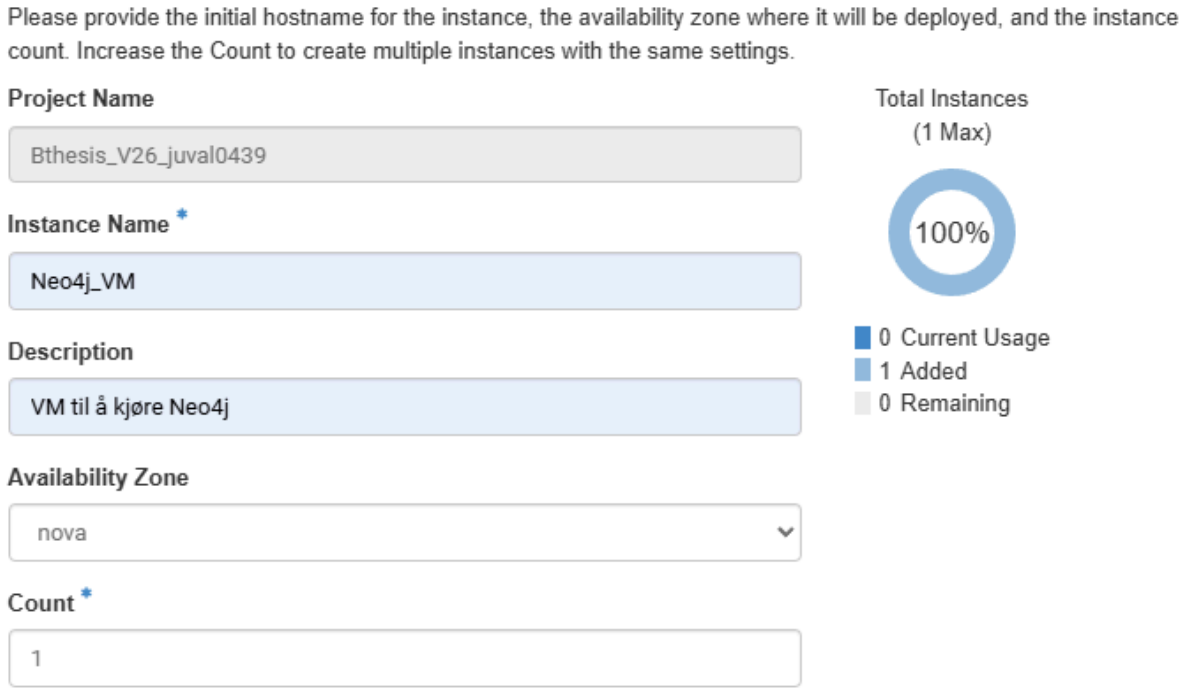
robsdedude. (u.å.). *neo4j-rust-driver: Hobby project: bolt driver for Neo4j written in Rust* [GitHub repository]. Hentet 24. mars 2026 fra https://github.com/robsdedude/neo4j-rust-driver

Wikipedia. (2026, 26. januar). *Politiets IT-enhet*. Hentet 17. april 2026 fra https://no.wikipedia.org/wiki/Politiets_IT-enhet

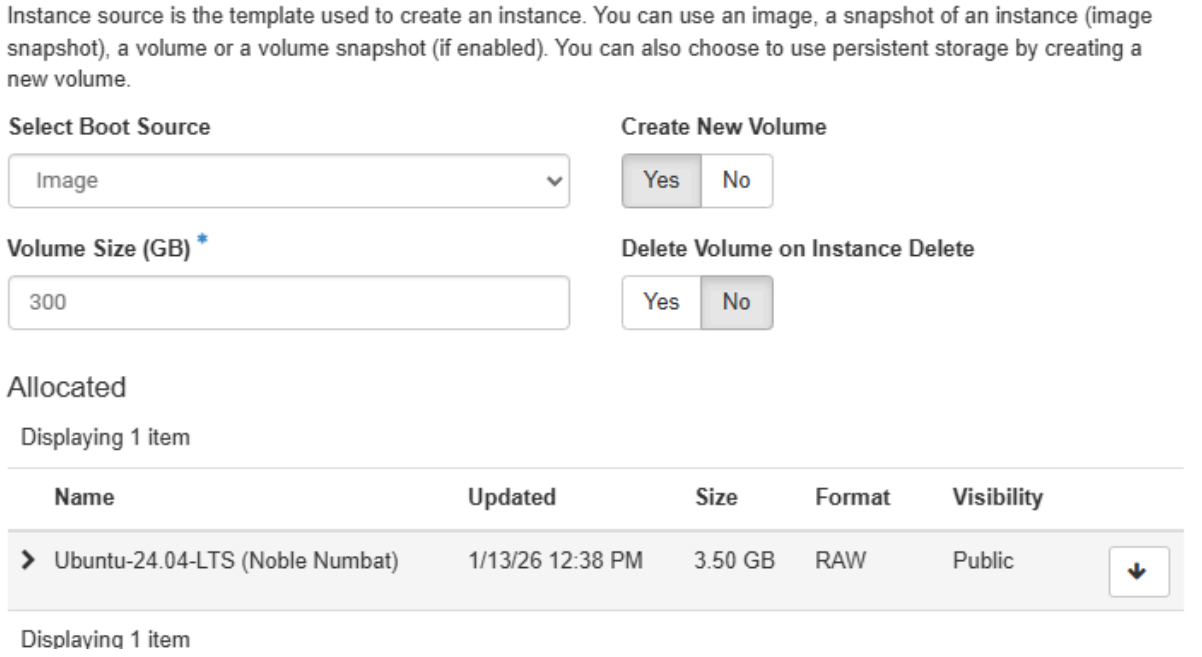
Vedlegg

Vedlegg 1: VM-oppsett..... 105
Vedlegg 2: Møtenotater.....112

Vedlegg 1: VM-oppsett



Vm beskrivelse



Volum størrelse på 300 GB og Ubuntu 24.04

Vi valgte flavor `aem.12c.64r.50g` som har 64 GB RAM, 12 kjerner og 50 GB Root Disk

Allocated

Displaying 1 item

Name	Type	Fingerprint
Neo4j_VM	ssh	d6:54:8c:1a:de:71:35:eb:b9:fd:5c:fd:56:18:10:e6

Displaying 1 item

La til alle SSH-nøklene til gruppen

Networks provide the communication channels for instances in the cloud. You can select ports instead of networks or a mix of both.

Allocated 1

Displaying 1 item

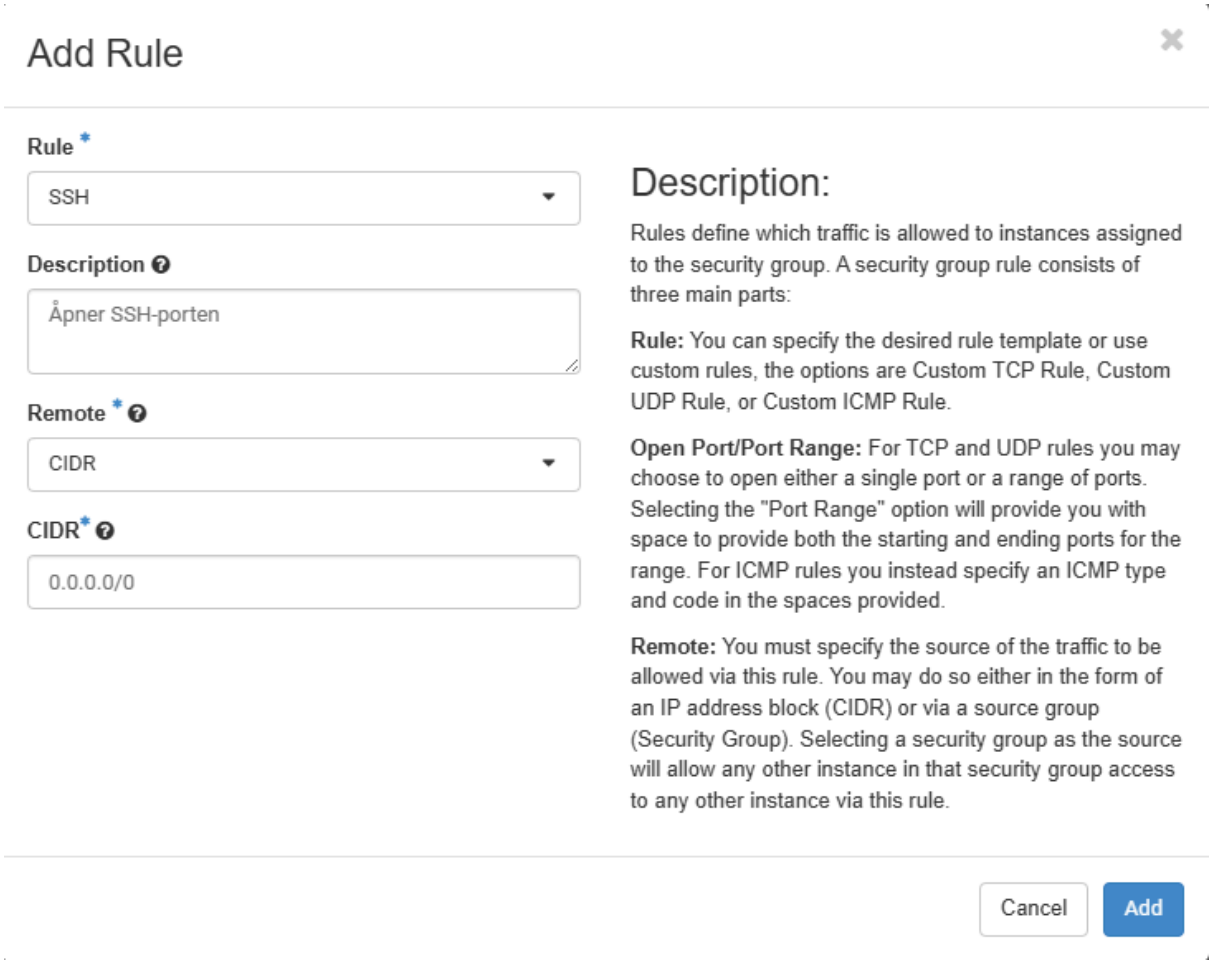
Network	Subnets Associated	Shared	Admin State	Status
oslomet	oslometv6 oslometv4	No	Up	Active

Displaying 1 item

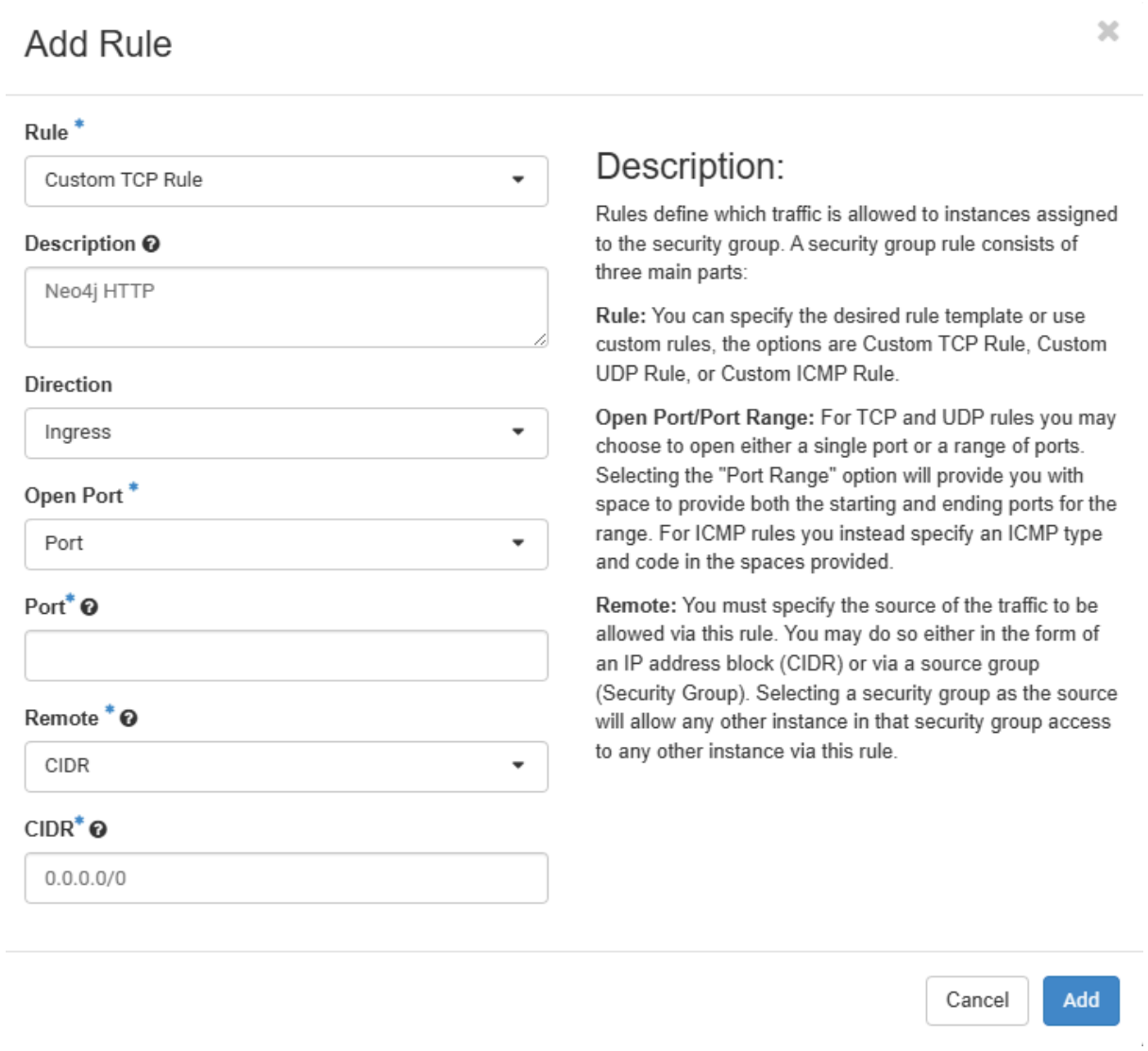
OsloMet Nettverket

```
NAME    ROTA TYPE  SIZE MOUNTPOIN
sda        1 disk  300G
├─sda1     1 part  299G /
├─sda14    1 part    4M
├─sda15    1 part  106M /boot/efi
└─sda16    1 part  913M /boot
```

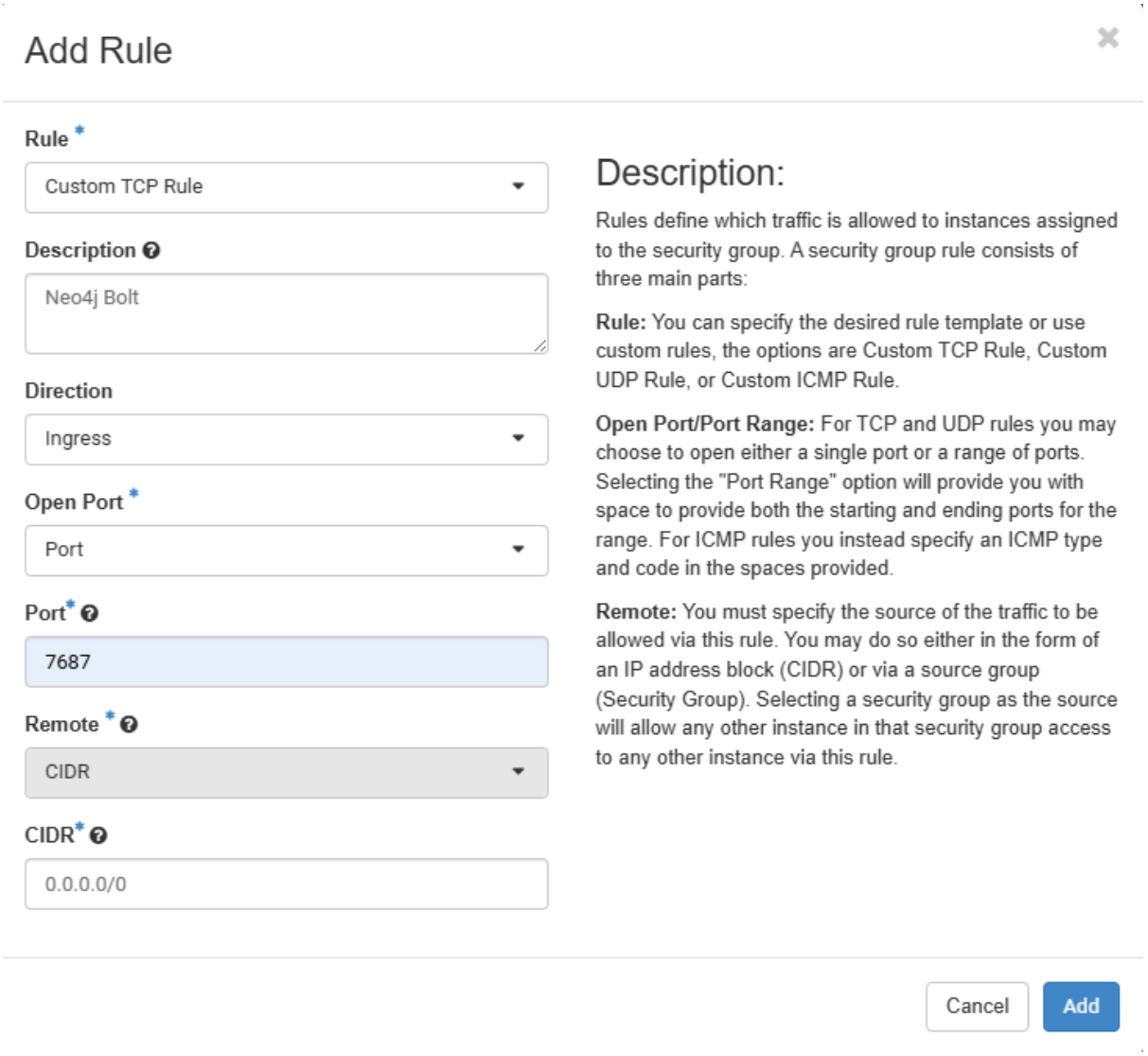
Det er 300 GB disk på VMen



La til regel for å åpne SSH-porten



La til regel for å åpne HTTP porten



La til regel for å åpne port 7687 som er Neo4j sin Bolt port

```
C:\Users\Admin2\.ssh>ssh -i id_ed25519 ubuntu@10.196.241.255
The authenticity of host '10.196.241.255 (10.196.241.255)' can't be established.
ED25519 key fingerprint is SHA256:19Qx+mOb7ZeAvy1NZBaUg19J7D0ukbTxFlkHnQWuyLM.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.196.241.255' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-90-generic x86_64)
```

Koble seg til VM

Man må være i .ssh mappen og deretter skrive inn
ssh -i id_ed25519 ubuntu@10.196.241.255
Første gang så må man skrive yes for å legge til VMen under Known hosts

```
ubuntu@neo4j-vm:~$ wget -O - https://debian.neo4j.com/neotechnology.gpg.key | sudo gpg --dearmor -o /usr/share/keyrings/neo4j.gpg
--2026-01-28 12:03:22--  https://debian.neo4j.com/neotechnology.gpg.key
Resolving debian.neo4j.com (debian.neo4j.com)... 18.173.5.47, 18.173.5.54, 18.173.5.103, ...
Connecting to debian.neo4j.com (debian.neo4j.com)|18.173.5.47|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3905 (3.8K) [application/pgp-keys]
Saving to: 'STDOUT'

-                           100%[=====================================================>]   3.81K  --.-KB/s    in 0s

2026-01-28 12:03:22 (71.5 MB/s) - written to stdout [3905/3905]

ubuntu@neo4j-vm:~$
```

Legge til Neo4j sitt repo

```
ubuntu@neo4j-vm:~$ echo 'deb [signed-by=/usr/share/keyrings/neo4j.gpg] https://debian.neo4j.com stable latest' | sudo tee /etc/apt/sources.list.d/neo4j.list
deb [signed-by=/usr/share/keyrings/neo4j.gpg] https://debian.neo4j.com stable latest
ubuntu@neo4j-vm:~$
```

```
ubuntu@neo4j-vm:~$ sudo apt update
```

Deretter kjøre “sudo apt update” for å oppdatere pakkelisten

```
ubuntu@neo4j-vm:~$ sudo apt install neo4j
```

Deretter kjører man “sudo apt install neo4j” for å installere neo4j

```
ubuntu@neo4j-vm:~$ sudo nano /etc/neo4j/neo4j.conf
```

Deretter kjører man denne kommandoen for å komme inn i neo4j sin config fil

```
#*****************************************************************
# Network connector configuration
#*****************************************************************

# With default configuration Neo4j only accepts local connections.
# To accept non-local connections, uncomment this line:
server.default_listen_address=0.0.0.0
```

Under Network connector configuration så uncommenter man “server.default_listen_adress=0.0.0.0

```
# Bolt connector
server.bolt.enabled=true
server.bolt.tls_level=DISABLED
server.bolt.listen_address=:7687
server.bolt.advertised_address=10.196.241.255:7687
```

Man må uncommente linjene under “Bolt connector” og legge til VMen sin IP og porten under “server.bolt.advertised_adress”

```
# HTTP Connector. There can be zero or one HTTP connectors.
server.http.enabled=true
server.http.listen_address=:7474
server.http.advertised_address=10.196.241.255:7474
```

Man må uncommente linjene under “HTTP Connector” og legge til VMen sin IP og porten under “server.bolt.advertised_adress”

```
# Java Heap Size: by default the Java heap size is dynamically calculated based
# on available system resources. Uncomment these lines to set specific initial
# and maximum heap size.
server.memory.heap.initial_size=16G
server.memory.heap.max_size=16G

# The amount of memory to use for mapping the store files.
# The default page cache memory assumes the machine is dedicated to running
# Neo4j, and is heuristically set to 50% of RAM minus the Java heap size.
server.memory.pagecache.size=40G
```

Deretter endrer man memory instillingene som at man kan utnytte mer av de 64 gb med RAM VMen har 16 Gb heap maks + 40 gb pagecache = 56 Gb som gir 8 Gb til overs til OS

Deretter lagrer man endringene og kan starte Neo4j

Koble seg til VM

```
C:\Windows\System32>ssh -i id_ed25519 ubuntu@10.196.241.255
Warning: Identity file id_ed25519 not accessible: No such file or directory.
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-90-generic x86_64)
```

Man må være i .ssh mappen og deretter skrive inn
ssh -i id_ed25519 ubuntu@10.196.241.255

Vedlegg 2: Møtenotater

Møtereferat 29.9.25 – Politi

Eva og Tage.

Første møte:

Gjennomgang av spørsmål, 4 fra forrige – good.
«Kan» kjøre i Asher, hvis server er problem. Sjekke med OsloMet om server derfra.

Character and coding, skal få info.

Se på dataen:

- Hvilke enheter.
- Sentralitet.
- Først: finne noder, starte med mennesker.
- Deretter: relasjoner.

Gjennomgang av datasettet:
Til Neste:

- Legge til Tage i GitHub.

- Pakke ut datasett.

- Lage en oversikt (Start) Hvilke noder? Hvilken type har nodene – slås sammen, hvor mange? Hvilke relasjoner – typer. Hvor mange av hver?

- Kanter - typer og antall.

- Få oppgaveteorem av Tage.

Fremover:

- Hvilken datamodell skal vi bruke?

- Skal få oppgave forslag av Tage.

Neste møte:

- Om to uker, 13-20 oktober.

Møtereferat 3.10.25

Julian og Scott.

Samtale med Torunn:

Kartlegge om oppgaven og databasen – case, hvem, hva, hvorfor osv...

Kan ikke være veileder før etter nyttår.

Vi har startet å kartlegge spørsmål vi skal stille Tage.
Skrevet i google.docs filen som er i Discord.

Database for stor for våre pc.

Møtereferat 13.10.26

Oppsummering fra forrige møte.

Hva kan vi gjøre:

Halvere? Forkorte?

Tage skal dele i mindre biter, 4.

Laster opp sånn at vi kan bruke det.

Ny fil (8 GB, eller lignende.)

Bruke dette fremover, inntil januar.

Referat 15.10

Fått tilsendt mindre pakker.

Åpnet og sett hvilke attributter som er mest brukt.
Begynt å se på de viktigste nodene.

Møte Januar:

- Definere mål?
- Kravspesifikasjon fra oppgradsgiver!! (Skriftlig) (Rammebetingelser osv).

- Plan for videre? (små steps, rekkefølge).
- Neste møte kl. 10 torsdag 22.01.25.

Møtereferat 23.10.25

Vi så litt nøyere på datasettet og fant 1 relasjon

HasShareholding og HasVotingRights kan bli kanter

Dele datasettet opp i flere store biter

600 Noder men det burde ha vært 1000 så er noe galt

Ekspertblikk på koden

Finne ut av hvordan nodene skal være koblet sammen

Politiet lager forslag til nodetyper og gjør det mer lesbart. Hvilke felter som bør være linker og noder Og hjelpe oss i gang

Vi skal se på koden og se på innlastingen.

Dele githubben med politiet som at de kan se på det og gi skrivetilgang.

Tages github Tagepoliti

Pushe koden til Munazza til github

Møte 21.1.26

Vil ha med alle properties. Alt som knytter noder sammen.

Sette på monitoreringsverktøy, se hvor mye den bruker. Se hvordan den vokser. Hvor mye tid som øker og størrelsen på ressurs.

Person med flere firma.

Politiet bruker datasettet?

Ressursbruk.

Bruke målinger.

Mulig videre:

Lage API som med spørring kan finne objekter uten å starte Neo4j, men inn i et nytt eget program? Fast API tilby API som gir URL, kode kaller database og gir svar. Oversikt i browser.

Test case.

Oppgave:

Et stort datasett - virkelig etterforskning øko kriminalitet. Stort og problematisk å bruke.

Analysere, lage god datamodell, lage kode for parsing og laste inn i grafdatabasen (neo4J , politi bruker). Evaluere lasting, datamodell, og ressursbruk i lasting og bruk av datasett. Og lage grensesnitt (API) som gjør det mulig for politiet å bruke det. Dokumentere alt.

Hva er formålet med prosjektet sett fra PIT sitt perspektiv?
Vil funnene våres bli brukt?

Hva skal vi gjøre konkret?

Et stort datasett - virkelig etterforskning øko kriminalitet. Stort og problematisk å bruke.

Analysere, lage god datamodell, lage kode for parsing og laste inn i grafdatabasen (neo4J , politi bruker). Evaluere lasting, datamodell, og ressursbruk i lasting og bruk av datasett. Og lage grensesnitt (API) som gjør det mulig for politiet å bruke det. Dokumentere alt.

Hva skal dere gjøre med dataen vi finner?

Evaluere implementasjon,

Reimplementere på nytt, bruke som mal.

Møtet 29. Januar:

Utbedre milepælsplan.

14 dagers plan og full plan - grundig.

ER-modell - Munazza har vel startet?

Firma - Firma. Moder firma?

Underfirma. Mange - mange: hjelpetabell.

Send link til datasettet og hva vi har gjort foreløpig til Torunn.

Møtet 06. Februar:

Se om kravspesifikasjon gir mening.

Ny milepælsplan skal være fra (dato) ikke uke.

Møtet 12. Februar:

Se om nodene har kommet inn riktig i Neo4j og kjør enkle Cypher-spørringer. (Fra tabell til tabell).

Person-statement-statement-person-statement-kilde-...

Møtet 26. Februar:

Trenger Webserver? apacheserver? snakke med Hedda? For API.

Spørre Tage om vi faktisk trenger webserver.

Spørre Hedda om det kanskje allerede er satt opp.

Til neste torsdag håper vi på godkjent fra Tage, og en satt opp webserver, skrive noe inn der.

Husk dokumentering av fastAPI. hva er det og hvordan funker det.

Møte 5. mars:

Skrive innholdsfortegnelse.

Fortsette uten godkjenning fra Tage.

Definere endepunkter for API selv. (Hvis ikke svar.)

Eventuelle flere ting å gjøre fra Arbeidsgiver fremover.

Lage bruksanvisning om hvordan gjøre om relasjon til graf database?

Til neste:

Få kontakt med Tage. Fortsette arbeid.

Lage plan for neste ukene av prosjektet. Konkrete mål - fra arbeidsgiver.

Kan også kontakte hun grafeksperten direkte.

Møte 12. mars med Tage:

Intro:

Mange linjer med kode.

De har problem med kjøring…

til FastAPI - ha med swagger, automatisk dokumentasjon.

Kalle personer/firma - få ut tilkoblede objekter og firmaer. personer.

Person til en eller flere firma , firma til en eller flere personer…

Firma - to objekter, liste med ownership og kontroll.

Person - to objekter, kan ha null, hva personen kontrollerer og en hva han eier.

Politiet kan videre legge ny relasjon ved personer knyttet til firma.

Ha med kode requests, teste at det virker, få JSON tilbake.

Starte på tekst:

Snakke med internveileder om mal. Har hun eller vise henne Tage sin.

Disposisjon - viktig. Bruke forslaget til Tage.

Starte med struktur. Det er poenget med malen.

Skrive bakfra og forover.

Skrive hva vi har gjort, notere mer underveis av prosess?

Vise at vi har lært.

Sette opp innholdsfortegnelsen og be Torunn se over.

Metode? Tage er imot.

Implementasjons kapittel. Teknisk, ikke praktisk.

“Hvordan”, viktig å svare på.

Men må fokus på å vise til sensor at det virker.

Ytelsesmålinger? - Tage synes det er viktig.

Anbefaler ikke demo på presentasjonen, men det er kult hvis det virker.

Kan lage relasjon mellom personer knyttet til samme firma. Gi relasjonen en tag (#)?

Kan eventuelt skrive om det. Fremtidige oppgraderinger.

Kort gjennomgang av dokumentene sendt fra oss.

Sjekke størrelse på databasen under kjøring.

Log in fikser politiet selv.

Men vi kan lage en egen løsning for ekstra arbeid.

(fastAPI har løsning på sin side)

Det kan vi også skrive om, og Tage kan gi hjelp om hvordan politiet gjør det.

Med log in kan vi ha logging - evt skrive om det.

Kan også sette opp logging (stang).

Politiet bruker egen logging - derfor.

Sende mail til Tage om hva som skjer videre etter møtet med Torunn.

Møte 13. mars med Torunn:

Planlegge fremover - fordele.

Neste møte 26. mars.

Torunn anbefaler å se på gamle oppgaver.

Sende mal fra Tage til Torunn.

Møte sammen neste uke?

Møte 9. April med Torunn:

Begynne med innholdsfortegnelsen og første kapittel

Lage oppsettet basert på hva vi har gjort (Tage sin er for gammel)

Spørre Tage om FastApi-endepunktene skal kobles til politiets brukergrensesnitt'

Skrivetips: Ikke nevn det samme flere ganger og begrunn hver påstand veldig godt.

Neste møte 16 april 10:00

Møte 16. April med Torunn:

Kanskje flytte på kravspekken?

Neste møte er fredag 24. April 10:00

Møte 24. April

Stikkord legg til neo4j.

Godt språk da:)

Bruk tegninger og illustrasjoner hvis mulig.

Kapittel 1 - legge til illustrasjoner. Tønne for VM?
Tydeliggjøre at vi parser json til neo4j - da kanskje med illustrasjoner.

Forklare størrelse på datasettet.
Forklare hvorfor de vil ha datasettet. Hvorfor vil politiet ha datasettet?

Kapittel 2 -

Endre navn til bakgrunn?

Rammeverk og verktøy - ikke under kap2.

Lage tabell, flytte kapittelene i vedlegg.

Kap 3 - Kravspekk

Ok,
Overordnet i rapport (evt).
Må være leselig. Se i canvas?

Videre:

Dele prosess inn i faser. Tydeliggjøre omfang, kode og prosess. Hvor mye gjorde vi faktisk?

Få input fra Tage!

Møte 30 April

Kapittel 1

Mer info om grafdatabasen pga datasettet er så stort, teoriene og bakgrunenn om hvorfor spørringene er lettere mot en grafdatabaaen enn relasjonsdatabase

Enda tydligere på hva som gjøres og hvorfor

"Politiet har et digert datasett" som er vanskelig å ha spørringer mot og det er bedre å konvertere det til graf

Kapittel 2

kanskje litt for mye

Mer figurer, flott kapittel

Tekniske ting kanskje under implementering

Kanskje komme med mer info om hva et API er

Få frem poenget ikke alle detaljer

Prosessdokumentasjon før produktdokumentasjon ?

Må kanskje bytte om litt på plasseringene av kapittlene men det skal gjøres helt til slutt

Møte 7. Mai

Ta bort den røde teksten!

Sette alt sammen!

Se om alt stemmer med inneholdsfortegnelsen

Sjekke om det er en rød tråd

Har man gjentatt seg selv?

Mange fine figurer!

Begrunne alt!

Nevne at vi har lært mye i refleksjonsdelen?

Kapittel 1 mye bedre

Legge til en setning om at politiet ønsket å få det inn i grafdatabasen og at politiet bruker python

Møte 13.mai

1.4.2 stavefeil - utfordrende

Problemstilling - ikke visualisering. Formulere den ut fra hva vi har gjort.

Kravspekk - kobles mot problemstilling

Avslutning - diskutere mer. Rundt problemstillingen og kravspekk?

Vedlegg - legge til og referere til dem i teksten. Levere som to zippa filer.

Presentasjonsøving 27. mai. P35 - PI857. Max 20min.